RAG Parsing and Chunking Notes

Structured Study Guide with Code Examples and Interview Questions

Table of Contents

1. Unstructured Parsing
2. Why Do We Need Unstructured Parsing?
3. Where Does Unstructured Parsing Fit In RAG?
4. What is Unstructured Data?
5. Challenges in Unstructured Parsing
6. Common Sources of Unstructured Data
7. Common Tools Used For Unstructured Parsing
8. Example Without Parsing
9. Example With Proper Parsing
10. Element-Based Parsing
11. Enterprise Example
12. Advantages of Unstructured Parsing
13. Limitations of Unstructured Parsing
14. Best Practices
15. Common Interview Questions
16. Interview Takeaway
17. Structured Parsing
18. Why Do We Need Structured Parsing?
19. Structured vs Unstructured Parsing
20. Where Does Structured Parsing Fit in RAG?
21. Common Structured Data Sources
22. Internal Working of Structured Parsing
23. Example: CSV Parsing
24. Example: JSON Parsing
25. Example: Database Parsing
26. Common Structured Parsing Formats
27. Structured Parsing Challenges
28. Structured Parsing Best Practices
29. Structured Parsing in Enterprise Systems
30. Advantages of Structured Parsing
31. Limitations of Structured Parsing
32. Structured Parsing Architecture
33. Enterprise Example
34. Common Interview Questions
35. Interview Takeaway
36. Metadata Extraction
37. Why Do We Need Metadata Extraction?
38. Metadata vs Content
39. Where Does Metadata Extraction Fit In RAG?
40. Types of Metadata
41. Common Metadata Fields

42. Metadata Extraction From PDFs
43. Metadata Extraction From CSV
44. Metadata Extraction From JSON
45. Explicit Metadata Extraction
46. Implicit Metadata Extraction
47. Automatic Metadata Extraction
48. Manual Metadata Extraction
49. Metadata Extraction Techniques
50. Metadata and Filtering
51. Metadata and Access Control
52. Metadata and Document Lifecycle
53. Metadata Extraction Challenges
54. Enterprise Example
55. Best Practices
56. Advantages of Metadata Extraction
57. Limitations
58. Common Interview Questions
59. Interview Takeaway
60. Content Cleaning
61. Why Do We Need Content Cleaning?
62. Where Does Content Cleaning Fit in RAG?
63. Common Sources of Dirty Data
64. Types of Content Cleaning
65. Common Cleaning Operations
66. Content Cleaning in PDFs
67. Content Cleaning in HTML
68. Content Cleaning in OCR
69. Why Content Cleaning Matters for Embeddings
70. Benefits for Retrieval
71. Content Cleaning Pipeline
72. Enterprise Example
73. Automated Cleaning Rules
74. TokenTextSplitter
75. Why Do We Need TokenTextSplitter?
76. Character vs Token Splitting
77. Why Is Token Splitting Important for LLMs?
78. Where Does TokenTextSplitter Fit In RAG?
79. Import
80. Installation
81. Basic Example
82. Core Parameters
83. Internal Working
84. Example Using Documents
85. Why Token Splitting Is Better For LLM Limits

86. TokenTextSplitter with OpenAI Models
87. TokenTextSplitter with Embedding Models
88. Example: Why Token Count Matters
89. Advantages
90. Limitations
91. Markdown Splitter (MarkdownHeaderTextSplitter)
92. Why Do We Need Markdown Splitter?
93. Why Is Markdown Common in RAG Systems?
94. Where Does Markdown Splitter Fit In RAG?
95. Import
96. Basic Markdown Example
97. Basic Example
98. Understanding headers_to_split_on
99. Output Structure
100. Metadata Generation
101. Why Metadata Matters?
102. Handling Multiple Header Levels
103. Internal Working
104. Markdown Splitter vs Character Splitter
105. Markdown Splitter vs RecursiveCharacterTextSplitter
106. Combining Markdown Splitter with Recursive Splitter
107. Enterprise Example
108. Common Use Cases
109. Advantages
110. Limitations
111. Best Practices
112. Common Interview Questions
113. Interview Takeaway
114. HTML Splitter (HTMLHeaderTextSplitter)
115. Why Do We Need HTML Splitter?
116. Why Is HTML Important in RAG?
117. Where Does HTML Splitter Fit In RAG?
118. Import
119. Basic HTML Example
120. Basic Example
121. Understanding headers_to_split_on
122. Output Structure
123. Metadata Generation
124. Multiple Header Levels
125. Internal Working
126. HTML Splitter vs CharacterTextSplitter
127. HTML Splitter vs RecursiveCharacterTextSplitter
128. Combining HTML Splitter with Recursive Splitting
129. Example: Documentation Website

130. Enterprise Use Cases
131. Advantages
132. Limitations
133. Best Practices
134. Common Interview Questions
135. Interview Takeaway
136. Code Splitter
137. Why Do We Need Code Splitter?
138. Why Is Code Chunking Different?
139. Where Does Code Splitter Fit In RAG?
140. Supported Languages
141. Import
142. Basic Example
143. What is Language-Aware Splitting?
144. Supported Language Enum
145. Python Code Example
146. Java Example
147. JavaScript Example
148. Internal Working
149. Why Code Structure Matters?
150. Chunking a Large Repository
151. Code Splitter vs CharacterTextSplitter
152. Code Splitter vs RecursiveCharacterTextSplitter
153. Code Embeddings and Chunking
154. Parent-Child Code Chunking
155. Enterprise Example
156. Common Use Cases
157. Advantages
158. Limitations
159. Best Practices
160. Common Interview Questions
161. Interview Takeaway
162. Semantic Chunking
163. Why Do We Need Semantic Chunking?
164. Why Is Semantic Chunking Important?
165. Traditional Chunking vs Semantic Chunking
166. Where Does Semantic Chunking Fit In RAG?
167. How Semantic Chunking Works
168. Internal Working
169. Example
170. Why Embeddings Are Used?
171. Semantic Similarity Concept
172. LangChain Semantic Chunking
173. Basic Example

174. Why Is It More Expensive?
175. Semantic Chunking vs RecursiveCharacterTextSplitter
176. Semantic Chunking vs Token Chunking
177. Real Enterprise Example
178. Benefits for Retrieval
179. Benefits for Embeddings
180. Common Use Cases
181. Advantages
182. Limitations
183. Best Practices
184. Common Interview Questions
185. Interview Takeaway
186. Chunk Size
187. Why is Chunk Size Important?
188. Where Does Chunk Size Fit in RAG?
189. Small Chunk Size Example
190. Large Chunk Size Example
191. Small vs Large Chunk Size
192. Why Chunk Size Impacts Retrieval
193. Why Chunk Size Impacts Embeddings
194. Chunk Size and Context Preservation
195. Impact on Vector Database
196. Chunk Size and Cost
197. Typical Chunk Sizes
198. Chunk Size with RecursiveCharacterTextSplitter
199. Chunk Size with TokenTextSplitter
200. Enterprise Example
201. Chunk Size Tuning
202. Common Mistakes
203. Chunk Size Best Practices
204. Chunk Size vs Chunk Overlap
205. Common Interview Questions
206. Interview Takeaway
207. Chunk Overlap
208. Why Do We Need Chunk Overlap?
209. Why Is Overlap Important in RAG?
210. Example Without Chunk Overlap
211. Example With Chunk Overlap
212. Where Does Overlap Fit?
213. How Chunk Overlap Works Internally
214. Overlap in RecursiveCharacterTextSplitter
215. Overlap in TokenTextSplitter
216. Benefits of Chunk Overlap
217. Example: Retrieval Without Overlap

218. Example: Retrieval With Overlap
219. Too Little Overlap
220. Too Much Overlap
221. Ideal Overlap
222. Chunk Size vs Chunk Overlap
223. Enterprise Example
224. Impact on Storage
225. Common Mistakes
226. Best Practices
227. Typical Production Settings
228. Common Interview Questions

Unstructured Parsing

What is Unstructured Parsing?

Unstructured Parsing is the process of extracting useful textual content from data sources that do not have a fixed schema, predefined structure, rows, columns, or predictable relationships.

In simple terms:

Structured data follows a clear format.

Example:

```
CSV
EmployeeID,Name,Department
101,John,Engineering
102,Alice,Finance
```

Unstructured data has no predefined structure.

Examples:

```
TEXT
PDFs
Word Documents
Emails
HTML Pages
Meeting Notes
Research Papers
Chat Messages
Policies
Contracts
Articles
```

Unstructured Parsing converts these raw files into machine-readable content that can be used by LangChain and LLM applications.

Why Do We Need Unstructured Parsing?

LLMs and Vector Databases work with text.

However, business content often exists in formats such as:

```
TEXT
PDF
DOCX
HTML
Email
Scanned Documents
Wiki Pages
Logs
```

These files may contain:

- Headers
- Footers
- Images
- Tables
- Page Numbers
- Signatures

- Formatting

Before chunking and embedding can begin, the system must determine:

```
TEXT
What content is useful?
What content should be ignored?
What content belongs together?
```

This is the responsibility of parsing.

Without Unstructured Parsing:

[X] Raw documents remain unusable.

[X] Noise enters embeddings.

[X] Retrieval quality decreases.

[X] Hallucinations increase.

With Unstructured Parsing:

[OK] Clean content extraction.

[OK] Better chunking.

[OK] Better embeddings.

[OK] Better retrieval quality.

Where Does Unstructured Parsing Fit In RAG?

```
TEXT
Raw Documents
(PDF/DOCX/HTML)
v
Unstructured Parsing
v
Clean Text Content
v
Metadata Extraction
v
Content Cleaning
v
Chunking
v
Embeddings
v
Vector Database
```

Unstructured Parsing is one of the earliest stages of the ingestion pipeline.

What is Unstructured Data?

Examples:

Example 1

PDF Document

```
TEXT
Employee Handbook

Page 1

Welcome to the company...

Page 2

Leave Policy...
```

No rows.

No columns.

No schema.

Example 2

Word Document

```
TEXT
Architecture Review

Introduction

System Design

Deployment Steps
```

Structure exists visually but not as a database schema.

Example 3

HTML Page

```
HTML
<h1>Authentication Guide</h1>

<p>JWT tokens are used...</p>
```

Useful information is mixed with HTML tags.

Challenges in Unstructured Parsing

Challenge 1: Mixed Content

Example PDF:

```
TEXT
Heading

Paragraph

Table

Image

Footer

Page Number
```

Parser must determine what is useful.

Challenge 2: Layout Complexity

Example:

```
TEXT
Column 1        Column 2
Data A          Data B
```

Simple text extraction may scramble content.

Challenge 3: Tables

Example:

Policy	Value
Password	90 Days

Many parsers struggle to preserve table structure.

Challenge 4: Images

Example:

```
TEXT
Architecture Diagram
```

Text parser cannot understand image content directly.

OCR may be required.

Challenge 5: Headers and Footers

Every page:

```
TEXT
Confidential

Page 1

Confidential

Page 2

Confidential
```

Repeated noise should be removed.

Common Sources of Unstructured Data

PDFs

Examples:

- Policies
- Contracts
- Research Papers

DOCX Files

Examples:

- SOPs
- Designs
- Manuals

HTML Files

Examples:

- Documentation
- Knowledge Bases

Emails

Examples:

- Support Requests
- Incident Reports

Chat Data

Examples:

- Slack
- Teams
- Customer Support

Scanned Documents

Examples:

- Invoices
- Signed Contracts

Common Tools Used For Unstructured Parsing

LangChain Loaders

Examples:

```
PYTHON
PyPDFLoader
PyMuPDFLoader
Docx2txtLoader
UnstructuredHTMLLoader
WebBaseLoader
```

Unstructured Framework

Very popular in enterprise RAG.

Can identify:

```
TEXT
Title
Paragraph
Table
Header
Footer
List
Image
```

instead of treating everything as plain text.

Example Without Parsing

Raw PDF Content:

```
TEXT
Confidential

Page 1

Security Policy

Passwords must be changed every 90 days.

Confidential

Page 2
```

Generated chunk:

```
TEXT
Confidential Page 1 Security Policy Passwords must be changed...
```

Poor quality.

Example With Proper Parsing

Parsed Content:

```
TEXT
Security Policy

Passwords must be changed every 90 days.
```

Much cleaner.

Element-Based Parsing

Modern parsing systems identify:

```
TEXT
Title

Paragraph

Table

List

Section
```

Example:

Document:

```
TEXT
Security Policy

Password Rules

MFA Requirements

Access Control
```

Parser can create:

```
TEXT
Title Element

Paragraph Element

List Element
```

instead of one giant block of text.

Benefits:

[OK] Better chunking

[OK] Better retrieval

[OK] Better context quality

Enterprise Example

Suppose a bank has:

```
TEXT
5000 PDFs
3000 DOCX
2000 HTML Files
```

Pipeline:

```
TEXT
Documents
v
Unstructured Parsing
v
Content Extraction
v
Cleaning
v
Chunking
v
Embeddings
v
Vector Database
```

Without parsing:

```
TEXT
Noise
Headers
Footers
Broken Tables
```

enter the index.

With parsing:

```
TEXT
Clean Business Content
```

enters the index.

Advantages of Unstructured Parsing

1. Works With Real Documents

Most enterprise content is unstructured.

2. Improves Retrieval Quality

Cleaner text leads to better search results.

3. Reduces Noise

Removes irrelevant information.

4. Better Chunk Generation

Chunks become semantically meaningful.

5. Supports Multiple Formats

PDFs, DOCX, HTML, Emails, etc.

Limitations of Unstructured Parsing

1. Parsing Errors

Complex documents may be parsed incorrectly.

2. Table Extraction Issues

Tables may lose formatting.

3. Scanned Documents Need OCR

Images cannot be understood directly.

4. Layout Complexity

Multi-column documents can be difficult.

Best Practices

[OK] Remove Headers and Footers

[OK] Validate Extracted Text

[OK] Preserve Logical Sections

[OK] Use Structure-Aware Parsers

[OK] Test Complex PDFs

[OK] Handle Tables Separately

Common Interview Questions

Q1. What is Unstructured Parsing?

Answer:

Unstructured Parsing is the process of extracting usable content from documents that do not follow a fixed schema, such as PDFs, DOCX files, HTML pages, contracts, emails, and research papers.

Q2. Why is Unstructured Parsing important in RAG systems?

Answer:

It converts raw documents into clean textual content, reducing noise and improving chunking, embeddings, retrieval quality, and final LLM responses.

Q3. What are examples of unstructured data?

Answer:

- PDFs
- Word Documents
- Emails
- HTML Pages
- Chat Messages
- Contracts
- Research Papers

Q4. What challenges exist in unstructured parsing?

Answer:

- Tables
- Images
- Headers
- Footers
- Multi-column layouts
- Scanned documents

Q5. What is element-based parsing?

Answer:

Element-based parsing identifies document components such as titles, paragraphs, tables, and lists instead of treating the whole document as one block of text.

Q6. How does unstructured parsing improve retrieval?

Answer:

It removes noise, preserves semantic meaning, and produces cleaner chunks, leading to higher retrieval accuracy.

Interview Takeaway

Unstructured Parsing is the process of extracting meaningful content from documents that do not follow a predefined schema. It is a foundational step in modern RAG systems because enterprise knowledge primarily exists in unstructured formats such as PDFs, Word documents, HTML pages, emails, and contracts. High-quality parsing significantly improves chunking, embedding generation, retrieval accuracy, and overall LLM response quality.

Structured Parsing

What is Structured Parsing?

Structured Parsing is the process of extracting information from data sources that already have a predefined schema, organized format, or predictable structure.

Unlike Unstructured Parsing, where the parser must determine what information is important, Structured Parsing works with data that already follows clear organizational rules.

Examples:

```
TEXT
CSV Files
Excel Files
JSON Files
Databases
API Responses
SQL Tables
Data Warehouses
ERP Records
CRM Records
```

In structured data:

- Rows have meaning.
- Columns have meaning.
- Relationships are defined.

- Field names are known.

Because of this, extraction becomes easier and more reliable.

Why Do We Need Structured Parsing?

Consider a CSV file:

```
CSV
EmployeeID,Name,Department
101,John,Engineering
102,Alice,Finance
103,David,HR
```

The parser already knows:

```
TEXT
EmployeeID -> Employee Identifier
Name -> Employee Name
Department -> Department Name
```

Unlike PDFs or DOCX files:

```
TEXT
No heading detection required
No layout analysis required
No footer removal required
No OCR required
```

The structure already exists.

Structured Parsing transforms this data into a format that can be consumed by LangChain and downstream RAG pipelines.

Structured vs Unstructured Parsing

Structured Parsing

Example:

```
CSV
ProductID,ProductName,Price
101,Laptop,50000
102,Mouse,1200
```

Characteristics:

[OK] Fixed schema

[OK] Consistent columns

[OK] Easy extraction

[OK] Predictable format

Unstructured Parsing

Example:

```
TEXT
Product Catalog

Laptop

Price: 50000

High-performance business laptop.
```

Characteristics:

[OK] No fixed schema

[OK] Requires content understanding

[OK] Requires section detection

[OK] Requires layout analysis

Where Does Structured Parsing Fit in RAG?

```
TEXT
Structured Data Source
v
Structured Parsing
v
Document Objects
v
Content Cleaning
v
Chunking
v
Embeddings
v
Vector Database
v
Retriever
v
LLM
```

Structured Parsing is usually simpler than Unstructured Parsing because data organization already exists.

Common Structured Data Sources

CSV Files

Example:

```
CSV
CustomerID,CustomerName,Region
101,John,India
102,Alice,USA
```

Excel Files

Example:

```
TEXT
Employees.xlsx
```

Contains:

- Name
- Department
- Location
- Salary

JSON Files

Example:

```
JSON
{
  "employee_id":101,
  "name":"John",
  "department":"Engineering"
}
```

Databases

Example:

```
SQL
SELECT
 employee_id,
 employee_name,
 department
FROM employees
```

APIs

Example:

```
JSON
{
  "id":101,
  "title":"Security Policy"
}
```

Internal Working of Structured Parsing

Suppose:

```
CSV
EmployeeID,Name,Department
101,John,Engineering
```

Parser performs:

Step 1

Read schema.

```
TEXT
EmployeeID
Name
Department
```

Step 2

Read row.

```
TEXT
101,John,Engineering
```

Step 3

Map fields.

```
TEXT
EmployeeID -> 101
Name -> John
Department -> Engineering
```

Step 4

Create Document.

```
PYTHON
Document(
    page_content="""
    Name: John
    Department: Engineering
    """
)
```

Example: CSV Parsing

Input:

```
CSV
TicketID,Issue,Priority
101,Payment Failed,High
```

Parsed Content:

```
TEXT
TicketID: 101

Issue: Payment Failed

Priority: High
```

Converted into:

```
PYTHON
Document(
    page_content="""
    TicketID: 101

    Issue: Payment Failed

    Priority: High
    """
)
```

Example: JSON Parsing

Input:

```
JSON
{
  "ticket_id":"T101",
  "priority":"High",
  "description":"Payment Failed"
}
```

Parsed Content:

```
TEXT
Payment Failed
```

Metadata:

```
PYTHON
{
    "ticket_id":"T101",
    "priority":"High"
}
```

Example: Database Parsing

Table:

```
TEXT
employees
```

```
TEXT
ID | Name | Department

101 | John | Engineering
```

Parser:

```
SQL
SELECT id,name,department
FROM employees
```

Output Document:

```
PYTHON
Document(
    page_content="""
    Name: John
    Department: Engineering
    """
)
```

Common Structured Parsing Formats

Row-Based Parsing

CSV:

```
TEXT
Row 1
Row 2
Row 3
```

Each row becomes a document.

Record-Based Parsing

JSON:

```
JSON
{
 "name":"John"
}
```

Each object becomes a document.

Table-Based Parsing

Database:

```
TEXT
Employees Table
```

Each record becomes a document.

Sheet-Based Parsing

Excel:

```
TEXT
Sheet1
Sheet2
Sheet3
```

Each row or sheet can become a document.

Structured Parsing Challenges

Structured data is easier than unstructured data, but challenges still exist.

Challenge 1: Missing Values

Example:

```
CSV
EmployeeID,Name,Department
101,John,
```

Missing:

```
TEXT
Department
```

Parser must handle null values.

Challenge 2: Inconsistent Data

Example:

```
CSV
101,John,Engineering

102,Alice,ENG

103,David,Engineering Team
```

Same department represented differently.

Challenge 3: Duplicate Records

Example:

```
CSV
101,John

101,John
```

Duplicates should be identified.

Challenge 4: Large Datasets

Example:

```
TEXT
10 Million Records
```

Loading everything simultaneously may be expensive.

Challenge 5: Schema Changes

Version 1:

```
TEXT
employee_name
```

Version 2:

```
TEXT
name
```

Parser must adapt.

Structured Parsing Best Practices

[OK] Validate Required Fields

Example:

```
TEXT
ID
Name
Department
```

Must exist.

[OK] Handle Null Values

Avoid creating empty documents.

[OK] Remove Duplicates

Improves retrieval quality.

[OK] Standardize Values

Convert:

```
TEXT
ENG
Engineering Team
Engineering
```

into:

```
TEXT
Engineering
```

[OK] Preserve Identifiers

Examples:

```
TEXT
Employee ID
Ticket ID
Order ID
Product ID
```

These are useful later.

Structured Parsing in Enterprise Systems

Examples:

CRM Systems

```
TEXT
Salesforce
HubSpot
Dynamics 365
```

Data:

```
TEXT
Leads
Customers
Deals
```

ERP Systems

```
TEXT
SAP
Oracle ERP
```

Data:

```
TEXT
Products
Orders
Finance
```

Database Systems

```
TEXT
MySQL
PostgreSQL
SQL Server
Oracle
```

Data Warehouses

```
TEXT
Snowflake
BigQuery
Redshift
```

Advantages of Structured Parsing

1. High Accuracy

Schema already exists.

2. Easier Validation

Fields are clearly defined.

3. Better Metadata Creation

IDs and categories are available.

4. Faster Processing

No layout analysis required.

5. Predictable Results

Structure remains consistent.

Limitations of Structured Parsing

1. Cannot Handle Rich Documents

Not suitable for:

```
TEXT
PDF
DOCX
HTML
```

2. Schema Dependency

Changes can break parsers.

3. Data Quality Issues

Bad source data produces bad documents.

4. Business Logic Complexity

Transformations may still be needed.

Structured Parsing Architecture

```
TEXT
CSV
JSON
Excel
Database
API
v
Structured Parsing
v
Field Mapping
v
Validation
v
Document Generation
v
Cleaning
v
Chunking
```

Enterprise Example

Suppose a bank stores:

```
TEXT
10 Million Customer Records
```

in:

```
TEXT
Snowflake
```

Pipeline:

```
TEXT
Snowflake
v
Structured Parsing
v
Validation
v
Document Creation
v
Chunking
v
Vector Database
```

Benefits:

[OK] Consistent ingestion

[OK] Scalable processing

[OK] Better retrieval quality

Common Interview Questions

Q1. What is Structured Parsing?

Answer:

Structured Parsing is the process of extracting information from data sources that already follow a predefined schema, such as CSV, JSON, Excel, databases, and API responses.

Q2. What are examples of structured data?

Answer:

- CSV Files
- JSON Files
- Excel Files
- Databases
- APIs
- Data Warehouses

Q3. Difference Between Structured and Unstructured Parsing?

Answer:

Structured Parsing works with predefined schemas and predictable fields, while Unstructured Parsing extracts information from documents such as PDFs, DOCX files, and HTML pages where structure is not explicitly defined.

Q4. Why is Structured Parsing easier?

Answer:

The schema already defines relationships, columns, and fields, reducing the need for layout analysis and content interpretation.

Q5. What are common challenges in Structured Parsing?

Answer:

- Missing Data
- Duplicate Records
- Schema Changes
- Inconsistent Values
- Large Datasets

Q6. How is Structured Parsing used in enterprise RAG systems?

Answer:

It is commonly used to ingest data from databases, CRM systems, ERP systems, APIs, and data warehouses before converting records into Documents.

Interview Takeaway

Structured Parsing is the process of extracting information from data sources with predefined schemas such as CSV files, JSON files, databases, Excel sheets, APIs, CRM systems, and ERP platforms. Because the structure is already known, extraction is highly accurate, predictable, and efficient compared to Unstructured Parsing. In enterprise RAG systems, Structured Parsing is commonly used for ingesting business records, customer information, transactions, products, tickets, and operational data before further processing through cleaning, chunking, and retrieval pipelines.

Metadata Extraction

What is Metadata Extraction?

Metadata Extraction is the process of identifying, collecting, and storing additional information about a document during ingestion.

Metadata is often described as:

```text
Data About Data
```

The document content contains the actual information.

Example:

```text
Passwords must be changed every 90 days.
```

Metadata describes that content.

Example:

```python
{
    "source": "security_policy.pdf",
    "page": 12,
    "department": "Security",
    "version": "2.0"
}
```

Metadata Extraction is one of the most important stages in enterprise RAG systems because it enables:

- Better Search
- Better Filtering

- Better Governance
- Better Traceability
- Better Access Control

Why Do We Need Metadata Extraction?

Consider two documents:

Document 1

```
TEXT
Password must be changed every 90 days.
```

Document 2

```
TEXT
Password must be changed every 60 days.
```

Without metadata:

```
TEXT
Which document is current?
Which department owns it?
Which version is correct?
```

Impossible to know.

With metadata:

```
PYTHON
{
    "department": "Security",
    "version": "3.0",
    "last_updated": "2025-08-01"
}
```

Now the document becomes traceable.

Metadata vs Content

Content

The actual information.

Example:

```
TEXT
Employees are entitled to 25 annual leave days.
```

Metadata

Information describing content.

Example:

```python
{
    "source":"hr_policy.pdf",
    "page":5,
    "department":"HR"
}
```

A common interview question:

```text
Content answers questions.

Metadata helps find the right content.
```

Where Does Metadata Extraction Fit In RAG?

```text
Document
v
Parsing
v
Metadata Extraction
v
Content Cleaning
v
Chunking
v
Embeddings
v
Vector Database
v
Retriever
```

Metadata is generated before chunking and indexing.

Types of Metadata

Metadata can come from multiple sources.

1. Source Metadata

Describes where the document came from.

Example:

```python
{
    "source":"employee_handbook.pdf"
}
```

Examples:

```
TEXT
Filename
Filepath
URL
Document Source
Database Table
```

2. Document Metadata

Describes characteristics of the document.

Example:

```
PYTHON
{
    "page":10,
    "language":"English",
    "document_type":"Policy"
}
```

3. Business Metadata

Describes business information.

Example:

```
PYTHON
{
    "department":"HR",
    "region":"India",
    "owner":"HR Team"
}
```

4. Technical Metadata

Generated during ingestion.

Example:

```
PYTHON
{
    "chunk_id":"123",
    "ingestion_date":"2026-08-15"
}
```

5. Security Metadata

Used for governance.

Example:

```
PYTHON
{
    "classification":"Confidential",
    "access_level":"Internal"
}
```

Common Metadata Fields

Enterprise systems frequently store:

```python
{
    "source": "...",
    "document_id": "...",
    "department": "...",
    "category": "...",
    "version": "...",
    "author": "...",
    "created_date": "...",
    "last_modified": "...",
    "region": "...",
    "language": "...",
    "classification": "...",
    "access_level": "..."
}
```

Metadata Extraction From PDFs

Suppose:

```text
Security_Policy.pdf
```

PyPDFLoader may produce:

```python
{
    "source":"Security_Policy.pdf",
    "page":5
}
```

Example:

```python
print(doc.metadata)
```

Output:

```python
{
    "source":"Security_Policy.pdf",
    "page":5
}
```

Metadata Extraction From CSV

CSV:

```csv
TicketID,Issue,Priority
101,Login Failed,High
```

Metadata:

```
PYTHON
{
    "source":"tickets.csv",
    "row":0
}
```

Additional metadata may be extracted:

```
PYTHON
{
    "ticket_id":"101",
    "priority":"High"
}
```

Metadata Extraction From JSON

JSON:

```
JSON
{
    "ticket_id":"T101",
    "priority":"High",
    "description":"Payment Failed"
}
```

Content:

```
TEXT
Payment Failed
```

Metadata:

```
PYTHON
{
    "ticket_id":"T101",
    "priority":"High"
}
```

Explicit Metadata Extraction

Example:

```
JSON
{
    "id":"DOC001",
    "department":"Security",
    "content":"Password Policy"
}
```

Code:

```python
Document(
    page_content=item["content"],
    metadata={
        "document_id": item["id"],
        "department": item["department"]
    }
)
```

Implicit Metadata Extraction

Some metadata is not directly stored.

Example filename:

```text
HR_LeavePolicy_India_2026.pdf
```

Extract:

```python
{
    "department":"HR",
    "region":"India",
    "year":"2026"
}
```

This is called derived metadata.

Automatic Metadata Extraction

Many loaders generate metadata automatically.

Examples:

TextLoader

```python
{
    "source":"policy.txt"
}
```

PyPDFLoader

```python
{
    "source":"policy.pdf",
    "page":4
}
```

WebBaseLoader

```python
{
    "source":"https://company.com/security"
}
```

CSVLoader

```
PYTHON
{
    "source":"tickets.csv",
    "row":0
}
```

Manual Metadata Extraction

Organizations frequently create custom metadata.

Example:

```
PYTHON
metadata={
    "document_type":"Policy",
    "department":"Security",
    "version":"3.0"
}
```

Benefits:

[OK] Better filtering

[OK] Better retrieval

[OK] Better governance

Metadata Extraction Techniques

Technique 1: File-Based Extraction

Example:

```
TEXT
HR_Policy_v3.pdf
```

Extract:

```
PYTHON
{
    "department":"HR",
    "version":"3"
}
```

Technique 2: Content-Based Extraction

Document:

```
TEXT
Department: Finance
```

Extract:

```
PYTHON
{
    "department":"Finance"
}
```

Technique 3: Database Extraction

Database:

```
TEXT
ID
Department
Owner
Version
```

Store fields directly as metadata.

Technique 4: API Extraction

API Response:

```
JSON
{
 "id":"123",
 "category":"Finance"
}
```

Metadata:

```
PYTHON
{
    "id":"123",
    "category":"Finance"
}
```

Metadata and Filtering

One of the biggest reasons for metadata extraction is filtering.

Example:

User asks:

```
TEXT
Show only HR policies.
```

Filter:

```
PYTHON
{
    "department":"HR"
}
```

Only HR documents are searched.

Metadata and Access Control

Example:

```
PYTHON
{
    "classification":"Confidential"
}
```

Only authorized users can access the content.

This is common in:

- Banking
- Healthcare
- Government
- Enterprise RAG

Metadata and Document Lifecycle

Metadata can track:

```python
{
    "version":"4.0",
    "status":"Active"
}
```

Benefits:

[OK] Identify outdated documents

[OK] Version control

[OK] Compliance tracking

Metadata Extraction Challenges

Challenge 1: Missing Metadata

Example:

```text
No document author available.
```

Challenge 2: Inconsistent Metadata

Example:

```text
HR
Human Resources
HumanResource
```

Need standardization.

Challenge 3: Conflicting Values

Document:

```text
Version 2
```

Filename:

```text
Version 3
```

Need validation.

Challenge 4: Excessive Metadata

Too many metadata fields:

```
PYTHON
100+ fields
```

Can increase storage and maintenance complexity.

Enterprise Example

Suppose a bank stores:

```
TEXT
Policies
Procedures
Compliance Guides
```

Metadata:

```
PYTHON
{
    "department":"Risk",
    "owner":"Compliance Team",
    "region":"India",
    "classification":"Internal",
    "version":"5.0"
}
```

Retrieval becomes significantly more accurate.

Best Practices

[OK] Store Source Information

```
PYTHON
source
```

[OK] Store Business Identifiers

```
PYTHON
document_id
ticket_id
employee_id
```

[OK] Standardize Metadata Values

Example:

```
TEXT
HR
```

instead of:

```
TEXT
HR
Human Resources
HumanResource
```

[OK] Include Security Metadata

```
PYTHON
classification
access_level
```

[OK] Keep Metadata Relevant

Avoid unnecessary fields.

Advantages of Metadata Extraction

1. Better Search

Documents become easier to locate.

2. Better Filtering

Department-based filtering.

3. Better Governance

Supports compliance.

4. Better Traceability

Identifies document sources.

5. Better Security

Supports access control.

Limitations

1. Requires Maintenance

Metadata standards must be managed.

2. Data Quality Issues

Incorrect metadata reduces retrieval quality.

3. Schema Changes

Metadata structures evolve.

Common Interview Questions

Q1. What is Metadata Extraction?

Answer:

Metadata Extraction is the process of collecting descriptive information about a document, such as source, author, department, version, category, and classification, during document ingestion.

Q2. Why is Metadata important in RAG systems?

Answer:

Metadata improves search quality, filtering, governance, traceability, and access control while helping retrieve the most relevant document.

Q3. What is the difference between content and metadata?

Answer:

Content contains the actual information, while metadata describes that information and provides contextual details.

Q4. What are common metadata fields in enterprise systems?

Answer:

- Source
- Document ID
- Department
- Category
- Author
- Version
- Created Date
- Classification
- Access Level

Q5. What is derived metadata?

Answer:

Derived metadata is metadata inferred from filenames, paths, content, or other sources rather than explicitly stored.

Q6. How does metadata improve retrieval?

Answer:

It enables filtering and narrowing search results, reducing irrelevant documents and improving retrieval precision.

Q7. Can metadata be used for security?

Answer:

Yes. Fields like classification and access level help enforce authorization and document governance.

Interview Takeaway

Metadata Extraction is the process of collecting descriptive information about a document during the ingestion pipeline. It is a critical component of enterprise RAG systems because metadata enables filtering, governance, traceability, compliance, access control, and retrieval optimization. High-quality metadata improves search accuracy, document management, and overall

system reliability, making it just as important as the document content itself.

Content Cleaning

What is Content Cleaning?

Content Cleaning is the process of removing unnecessary, noisy, invalid, redundant, or irrelevant information from a document before it is sent for chunking, embedding generation, and indexing.

The primary goal of Content Cleaning is:

```
TEXT
Keep useful information
Remove useless information
```

Raw documents often contain many elements that add no value to retrieval systems.

Examples:

- Headers
- Footers
- Page Numbers
- Navigation Menus
- Advertisements
- Empty Spaces
- Special Characters
- Duplicate Text
- Boilerplate Text
- Legal Disclaimers

Content Cleaning improves the quality of downstream RAG components.

Why Do We Need Content Cleaning?

Consider a PDF:

```
TEXT
Company Security Policy

Page 1

Company Confidential

Passwords must be changed every 90 days.

Company Confidential

Page 2

Multi-factor authentication is mandatory.
```

Without Cleaning:

```
TEXT
Company Security Policy

Page 1

Company Confidential

Passwords must be changed every 90 days.

Company Confidential

Page 2
```

The embedding model learns:

```
TEXT
Company Confidential
Page 1
Page 2
```

which adds no value.

After Cleaning:

```
TEXT
Passwords must be changed every 90 days.

Multi-factor authentication is mandatory.
```

Now embeddings focus on meaningful content.

Where Does Content Cleaning Fit in RAG?

```
TEXT
Document
v
Parsing
v
Metadata Extraction
v
Content Cleaning
v
Content Normalization
v
Chunking
v
Embeddings
v
Vector Database
```

Cleaning happens before chunking.

This ensures only high-quality content enters the retrieval pipeline.

Common Sources of Dirty Data

PDFs

Examples:

```
TEXT
Page Numbers

Headers

Footers

Watermarks
```

DOCX Files

Examples:

```
TEXT
Repeated Titles

Revision History

Comments
```

Websites

Examples:

```
TEXT
Navigation Menus

Cookie Notices

Advertisements

Sidebar Links
```

OCR Documents

Examples:

```
TEXT
Spelling Errors

Broken Words

Incorrect Characters
```

Logs

Examples:

```
TEXT
Timestamps

Debug Information

System Noise
```

Types of Content Cleaning

1. Header Removal

Example:

Raw:

```
TEXT
Security Policy

Page 1

Security Policy

Page 2

Security Policy

Page 3
```

Cleaned:

```
TEXT
Passwords must be changed every 90 days.

MFA is required.
```

2. Footer Removal

Raw:

```
TEXT
Passwords must be changed every 90 days.

Company Confidential
```

Cleaned:

```
TEXT
Passwords must be changed every 90 days.
```

3. Page Number Removal

Raw:

```
TEXT
Page 1

Security Guidelines

Page 2

Password Rules
```

Cleaned:

```
TEXT
Security Guidelines

Password Rules
```

4. Whitespace Cleaning

Raw:

```
TEXT
Password     Policy


MFA     Required
```

Cleaned:

```
TEXT
Password Policy

MFA Required
```

5. Duplicate Content Removal

Raw:

```
TEXT
Password Policy

Password Policy

Password Policy
```

Cleaned:

```
TEXT
Password Policy
```

6. Navigation Removal

Website Content:

```
TEXT
Home

Products

About Us

Contact

Security Policy
```

Cleaned:

```
TEXT
Security Policy
```

7. Advertisement Removal

Raw:

```
TEXT
Security Policy

Buy Now

Get Discount

Password Rules
```

Cleaned:

```
TEXT
Security Policy

Password Rules
```

8. Boilerplate Removal

Raw:

```
TEXT
All Rights Reserved.

Company Confidential.

Passwords must be changed every 90 days.
```

Cleaned:

```
TEXT
Passwords must be changed every 90 days.
```

9. Empty Line Removal

Raw:

```
TEXT
Password Policy




MFA Required
```

Cleaned:

```
TEXT
Password Policy

MFA Required
```

10. Special Character Cleaning

Raw:

```
TEXT
Password Policy #### $$$ ***
```

Cleaned:

```
TEXT
Password Policy
```

Common Cleaning Operations

Remove Extra Spaces

Before:

```
TEXT
Password      Policy
```

After:

```
TEXT
Password Policy
```

Remove Excessive Newlines

Before:

```
TEXT
Password Policy




MFA
```

After:

```
TEXT
Password Policy

MFA
```

Remove HTML Tags

Before:

```
HTML
<h1>Password Policy</h1>
```

After:

```
TEXT
Password Policy
```

Remove Unwanted Symbols

Before:

```
TEXT
Password Policy $$$$
```

After:

```
TEXT
Password Policy
```

Content Cleaning in PDFs

Common Noise:

```
TEXT
Page Numbers

Headers

Footers

Legal Notices

Watermarks
```

Example:

Before:

```
TEXT
Page 5

Company Confidential

Passwords must be changed every 90 days.
```

After:

```
TEXT
Passwords must be changed every 90 days.
```

Content Cleaning in HTML

Raw HTML Extraction:

```
TEXT
Home

Products

Contact

Password Policy
```

Cleaned:

```
TEXT
Password Policy
```

Navigation menus are removed.

Content Cleaning in OCR

Raw OCR:

```
TEXT
Passvvord poIicy

MFA reqiired
```

Potentially Corrected:

```
TEXT
Password policy

MFA required
```

Cleaning improves readability before indexing.

Why Content Cleaning Matters for Embeddings

Embedding Models learn patterns from text.

Suppose:

Raw Text:

```
TEXT
Page 1

Confidential

Password Policy

Page 2

Confidential
```

Embedding learns:

```
TEXT
Page
Confidential
```

many times.

This wastes embedding capacity.

Clean Text:

```
TEXT
Password Policy
```

The embedding focuses only on business meaning.

Benefits for Retrieval

Without Cleaning:

User asks:

```
TEXT
What is the password policy?
```

Retriever may match:

```
TEXT
Confidential

Page 1
```

instead of actual content.

With Cleaning:

Retriever matches:

```
TEXT
Passwords must be changed every 90 days.
```

Improving accuracy.

Content Cleaning Pipeline

```
TEXT
Raw Document
v
Parse Content
v
Remove Headers
v
Remove Footers
v
Remove Page Numbers
v
Remove Duplicates
v
Remove Empty Lines
v
Remove Boilerplate
v
Clean Content
```

Enterprise Example

Suppose a company has:

```
TEXT
10,000 PDFs

5,000 DOCX Files

20,000 HTML Pages
```

Without Cleaning:

```
TEXT
Headers
Footers
Page Numbers
Legal Notices
Navigation Menus
```

enter embeddings.

With Cleaning:

```
TEXT
Business Content Only
```

This significantly improves retrieval quality.

Automated Cleaning Rules

Typical rules:

```
TEXT
Remove repeated headers

Remove repeated


# Content Normalization

## What is Content Normalization?

Content Normalization is the process of transforming cleaned content into a consistent,
standardized, and machine-friendly format before chunking, embedding, and indexing.

The goal is:
```

Same Meaning v Same Representation

```
TEXT

Content Cleaning removes noise.

Content Normalization standardizes the remaining content.

Example:

Before:
```

HR Human Resources Human Resource Dept

```
TEXT

After:
```

Human Resources

```
TEXT

All variations now have the same representation.

---

# Why Do We Need Content Normalization?

Enterprise data comes from multiple sources:
```

PDFs DOCX HTML Databases APIs Emails Excel Files

```
TEXT

Different systems often represent the same information differently.

Example:

Source A
```

USA

```
TEXT

Source B
```

United States

```
TEXT

Source C
```

United States of America

```
TEXT

Without normalization:
```

3 Different Values

```
TEXT

With normalization:
```

United States

```
TEXT

This improves consistency across the knowledge base.

---

# Content Cleaning vs Content Normalization

## Content Cleaning

Removes noise.

Before:
```

Page 1

Password Policy

Confidential

```
TEXT

After:
```

Password Policy

```
TEXT

---

## Content Normalization

Standardizes information.

Before:
```

Pwd Policy Password Policy Pwd. Policy

```
TEXT

After:
```

Password Policy

```
TEXT

---

# Where Does Content Normalization Fit In RAG?
```

Document v Parsing v Metadata Extraction v Content Cleaning v Content Normalization v Chunking v Embeddings v Vector Database

```
TEXT

Normalization happens before chunking and embedding.

---

# Why Normalization Improves RAG?

Suppose documents contain:

Document 1
```

MFA Required

```
TEXT

Document 2
```

Multi-Factor Authentication Required

```
TEXT

Document 3
```

Multi Factor Authentication Required

```
TEXT

Without normalization:
```

Different Representations

```
TEXT

With normalization:
```

Multi-Factor Authentication Required

```
TEXT

Consistency improves retrieval and search quality.

---

# Common Content Normalization Operations

## 1. Case Normalization

Before:
```

PASSWORD POLICY

Password Policy

password policy

```
TEXT

After:
```

password policy

```
TEXT

or
```

Password Policy

```
TEXT

depending on rules.

---

## 2. Whitespace Normalization

Before:
```

Password Policy

```
TEXT

After:
```

Password Policy

```
TEXT

---

## 3. Date Normalization

Before:
```

01-01-2026

2026/01/01

January 1, 2026

```
TEXT

After:
```

2026-01-01

```
TEXT

Standardized ISO format.

---

## 4. Number Normalization

Before:
```

1000

1,000

1K

```
TEXT

After:
```

1000

```
TEXT

---

## 5. Currency Normalization

Before:
```

Rs.1000 INR 1000 INR 1000

```
TEXT

After:
```

INR 1000

```
TEXT

---

## 6. Abbreviation Expansion

Before:
```

HR MFA OTP SSO

```
TEXT

After:
```

Human Resources

Multi-Factor Authentication

One-Time Password

Single Sign-On

```
TEXT

---

## 7. Synonym Standardization

Before:
```

Employee

Staff

Associate

```
TEXT

After:
```

Employee

```
TEXT

if business rules require it.

---

## 8. Unit Standardization

Before:
```

10 KM

10 Kilometers

10000 Meters

```
TEXT

After:
```

10 Kilometers

```
TEXT

---

## 9. Boolean Normalization

Before:
```

Yes YES Y True 1

```
TEXT

After:
```

True

```
TEXT

---

## 10. Country Normalization

Before:
```

USA US United States United States of America

```
TEXT

After:
```

United States

```
TEXT

---

# Text Normalization

Example:

Before:
```

PASSWORD POLICY

PASSWORDS MUST CHANGE EVERY 90 DAYS

```
TEXT

After:
```

Password Policy

Passwords must change every 90 days

```
TEXT

More readable and consistent.

---

# Email Normalization

Before:
```

John.Doe@Company.Com

```
TEXT

After:
```

john.doe@company.com

```
TEXT

Useful for identity matching.

---

# Phone Number Normalization

Before:
```

9876543210

+91 9876543210

+91-9876543210

```
TEXT

After:
```

+919876543210

```
TEXT

---

# Address Normalization

Before:
```

Bangalore

Bengaluru

B'lore

```
TEXT

After:
```

Bengaluru

```
TEXT

---

# OCR Normalization

OCR often introduces errors.

Before:
```

Passvword Policy

```
TEXT

After:
```

Password Policy

```
TEXT

Normalization improves readability.

---

# Structured Data Normalization

Example CSV:
```

Department

HR Human Resources HumanResource

```
TEXT

After:
```

Department

Human Resources Human Resources Human Resources

```
TEXT

Consistency improves retrieval quality.

---

# Unstructured Data Normalization

Example PDF:
```

Pwd Policy

Multi Factor Auth

```
TEXT

After:
```

Password Policy

Multi-Factor Authentication

```
TEXT

---

# Linguistic Normalization

Sometimes words are normalized to their root form.

Before:
```

running runs ran

```
TEXT

After:
```

run

```
TEXT

This helps improve search consistency.

---

# Entity Normalization

Before:
```

MSFT Microsoft Microsoft Corporation

```
TEXT

After:
```

Microsoft

```
TEXT

Useful in enterprise knowledge systems.

---

# Data Quality Benefits

Without Normalization:
```

HR

Human Resources

HumanResource

```
TEXT

Three values.

With Normalization:
```

Human Resources

```
TEXT

One value.

Benefits:

[OK] Consistency

[OK] Better Search

[OK] Better Filtering

[OK] Better Analytics

---

# Enterprise Example

Suppose:
```

50,000 Documents

```
TEXT

Collected from:
```

PDFs Emails CRM SharePoint APIs Databases

```
TEXT

Different teams use:
```

HR Human Resources HR Team

```
TEXT

Normalization Pipeline:
```

Raw Content v Cleaning v Normalization v Standard Vocabulary v Chunking v Embeddings

```
TEXT

Results in a consistent enterprise knowledge base.

---

# Content Normalization Pipeline
```

Raw Content v Case Normalization v Whitespace Normalization v Date Standardization v Number Standardization v Abbreviation Expansion v Entity Standardization v Normalized Content

```
TEXT

---

# Benefits for Embeddings

Without Normalization:
```

MFA

Multi-Factor Authentication

Multi Factor Authentication

```
TEXT

Multiple embeddings may be created.

With Normalization:
```

Multi-Factor Authentication

```
TEXT

More consistent semantic representation.

---

# Benefits for Retrieval

User Query:
```

What are the MFA requirements?

```
TEXT

Document Contains:
```

Multi-Factor Authentication

```
TEXT

Normalization helps create stronger semantic alignment.

---

# Challenges in Content Normalization

## Challenge 1: Over-Normalization

Before:
```

Java JavaScript

```
TEXT

After:
```

Programming Language

```
TEXT

Meaning lost.

---

## Challenge 2: Business Context

Example:
```

Associate Employee Contractor

```
TEXT

May not mean the same thing.

---

## Challenge 3: Domain-Specific Terms

Healthcare:
```

BP

```
TEXT

Could mean:
```

Blood Pressure

```
TEXT

or something else in another domain.

---

## Challenge 4: Acronym Ambiguity

Example:
```

AI

```
TEXT

Could represent multiple meanings depending on context.

---

# Best Practices

### [OK] Normalize Dates

### [OK] Normalize Numbers

### [OK] Standardize Departments

### [OK] Standardize Business Terms

### [OK] Handle OCR Corrections

### [OK] Preserve Important Meaning

### [OK] Avoid Over-Normalization

### [OK] Create Domain Dictionaries

---

# Advantages

### 1. Better Consistency

Documents follow common standards.

---

### 2. Better Retrieval

Improves matching.

---

### 3. Better Filtering

Standardized values improve filtering.

---

### 4. Better Embeddings

Reduces semantic fragmentation.

---

### 5. Better Analytics

Reporting becomes easier.

---

# Limitations

### 1. Requires Business Rules
```

Normalization logic must be defined.

---

2. Risk of Meaning Loss

Over-normalization can remove distinctions.

---

3. Domain Dependency

Different industries require different rules.

Common Interview Questions

Q1. What is Content Normalization?

Answer:

Content Normalization is the process of transforming cleaned content into a standardized and consistent format before chunking, embedding, and indexing.

Q2. Why is Content Normalization important in RAG systems?

Answer:

It ensures consistent representation of information, improving embedding quality, retrieval accuracy, filtering, and overall search performance.

Q3. What is the difference between Content Cleaning and Content Normalization?

Answer:

Content Cleaning removes noise and irrelevant data, while Content Normalization standardizes the remaining content into a consistent representation.

Q4. What are common normalization operations?

Answer:

- Case Normalization
- Date Normalization
- Number Normalization
- Currency Normalization
- Entity Normalization
- Abbreviation Expansion
- Whitespace Normalization

Q5. How does normalization improve retrieval?

```
Answer:

It ensures similar concepts are represented consistently, making it easier for retrievers
and embedding models to identify related content.

---

## Q6. What is over-normalization?

Answer:

Over-normalization occurs when important distinctions or business meaning are lost because
content is standardized too aggressively.

---

## Q7. What are examples of entity normalization?

Answer:
```

Microsoft MSFT Microsoft Corporation

```
TEXT

becoming:
```

Microsoft

```
TEXT

---

# Interview Takeaway

Content Normalization is the process of transforming cleaned content into a standardized,
consistent, and machine-friendly format before chunking and indexing. Common normalization
operations include standardizing dates, numbers, currencies, abbreviations, entities,
departments, and business terminology. In enterprise RAG systems, normalization improves
consistency, retrieval accuracy, embedding quality, filtering, governance, and overall
search performance while reducing semantic fragmentation across large document collections.


# CharacterTextSplitter

## What is CharacterTextSplitter?

CharacterTextSplitter is one of the simplest text splitters available in LangChain.

It divides text into chunks based on a fixed number of characters.

The splitter does not understand:

- Meaning
- Sentences
- Paragraphs
- Topics
- Semantic Relationships

Instead, it simply counts characters and creates chunks according to a specified chunk size.
```

```
Example:

Original Text:
```

Passwords must be changed every 90 days.

Multi-factor authentication is required.

Accounts are locked after five failed login attempts.

```
TEXT

CharacterTextSplitter may create:

Chunk 1
```

Passwords must be changed every 90 days.

Multi-fa

```
TEXT

Chunk 2
```

ulti-factor authentication is required.

Accou

```
TEXT

Notice:

[X] Meaning can be broken.

[X] Sentences can be split.

[OK] Simple to implement.

[OK] Fast.

---

# Why Do We Need CharacterTextSplitter?

Suppose a document contains:
```

50000 Characters

```
TEXT

Embedding models have limits.

The document cannot be embedded as one large block.

CharacterTextSplitter helps by dividing content into smaller chunks.

Example:
```

50000 Characters v 1000 Characters Per Chunk v 50 Chunks

```
TEXT

Now every chunk can be embedded independently.

---

# Where Does CharacterTextSplitter Fit In RAG?
```

Document v Parsing v Cleaning v Normalization v CharacterTextSplitter v Chunks v Embeddings v Vector Database

```
TEXT

---

# Import
```

from langchain_text_splitters import CharacterTextSplitter

```
TEXT

---

# Basic Example
```

from langchain_text_splitters import CharacterTextSplitter

text = """ Passwords must be changed every 90 days.

Multi-factor authentication is required.

VPN access requires approval. """

splitter = CharacterTextSplitter(chunk_size=50, chunk_overlap=0)

chunks = splitter.split_text(text)

print(chunks)

```
TEXT

Output:
```

['Passwords must be changed every 90 days...', 'Multi-factor authentication is required...', 'VPN access requires approval...']

```
TEXT

---

# Core Parameters

## chunk_size

Determines:
```

Maximum number of characters per chunk

```
TEXT

Example:
```

CharacterTextSplitter(chunk_size=1000)

```
TEXT

Means:
```

1000 characters per chunk

```
TEXT

---

## chunk_overlap

Determines:
```

How many characters overlap between chunks

```
TEXT

Example:
```

CharacterTextSplitter(chunk_size=100, chunk_overlap=20)

```
TEXT

Result:
```

Chunk 1 ABCDE...

Chunk 2 (last 20 chars repeated)

```
TEXT

Used to preserve context.

---

## separator

Controls where splitting occurs.

Example:
```

CharacterTextSplitter(separator="\n")

```
TEXT

Split on line breaks.

---

# Example Without Overlap

Text:
```

ABCDEFGHIJ

```
TEXT

Configuration:
```

chunk_size=4 chunk_overlap=0

```
TEXT

Output:
```

ABCD

EFGH

IJ

```
TEXT

No shared context.

---

# Example With Overlap

Text:
```

ABCDEFGHIJ

```
TEXT

Configuration:
```

chunk_size=4 chunk_overlap=2

```
TEXT

Output:
```

ABCD

CDEF

EFGH

GHIJ

```
TEXT

Context carried forward.

---

# Working with Documents

Most LangChain applications use:
```

Document objects

```
TEXT

Example:
```

from langchain_core.documents import Document from langchain_text_splitters import CharacterTextSplitter

docs = [Document(page_content=""" Password policy details... """)]

splitter = CharacterTextSplitter(chunk_size=100, chunk_overlap=20)

chunks = splitter.split_documents(docs)

```
TEXT

Output:
```

List of chunked Documents

```
TEXT

---

# Internal Working

Suppose:
```

1000 character document

```
TEXT

Configuration:
```

chunk_size = 250 chunk_overlap = 50

```
TEXT

Process:
```

Characters 1-250

Characters 201-450

Characters 401-650

Characters 601-850

Characters 801-1000

```
TEXT

Repeated context preserves continuity.

---

# CharacterTextSplitter with PDFs

Pipeline:
```

PDF v PyPDFLoader v Document v CharacterTextSplitter v Chunks v Embeddings

```
TEXT

Example:
```

loader = PyPDFLoader("security_policy.pdf")

docs = loader.load()

splitter = CharacterTextSplitter(chunk_size=1000, chunk_overlap=100)

chunks = splitter.split_documents(docs)

```
TEXT

---

# CharacterTextSplitter with CSV
```

loader = CSVLoader("tickets.csv")

docs = loader.load()

splitter = CharacterTextSplitter(chunk_size=500)

chunks = splitter.split_documents(docs)

```
TEXT

---

# Advantages

## 1. Easy To Understand

Very beginner friendly.

---

## 2. Fast

Simple character counting.

---

## 3. Lightweight

Minimal processing overhead.

---

## 4. Good For Simple Text

Works well for:

- Logs
- Notes
- Small Documents

---

## 5. Configurable

Supports:

- chunk_size
- overlap
- separator

---

# Limitations

## 1. No Semantic Awareness

Example:
```

Sentence starts here...

```
TEXT

May split midway.

---

## 2. Breaks Sentences

Example:
```

Passwords must be cha nged every 90 days

```
TEXT

Context quality decreases.

---

## 3. Not Ideal For RAG

Modern RAG systems usually prefer:
```

RecursiveCharacterTextSplitter

```
TEXT

---

## 4. Poor Context Preservation

Only counts characters.

Does not understand:

- Words
- Sentences
- Paragraphs

---

# CharacterTextSplitter vs RecursiveCharacterTextSplitter

## CharacterTextSplitter

Splits:
```

By Character Count

```
TEXT

Pros:

[OK] Simple

[OK] Fast

Cons:

[X] Sentence breaks

[X] Paragraph breaks

---

## RecursiveCharacterTextSplitter

Splits:
```

Paragraph v Sentence v Word

```
TEXT

Pros:

[OK] Better chunk quality

[OK] Better retrieval

[OK] Better RAG performance

Cons:

[X] Slightly more processing

---

# Enterprise Use Cases

## Log Files

Example:
```

Application Logs Server Logs Audit Logs

```
TEXT

---

## Simple Text Documents

Example:
```

Policies Guidelines Notes

```
TEXT

---

## Small Knowledge Bases

Example:
```

Internal FAQs

```
TEXT

---

# Common Interview Questions

## Q1. What is CharacterTextSplitter?

Answer:

CharacterTextSplitter is a LangChain text splitter that divides content into chunks based on
 a fixed number of characters.

---

## Q2. What parameters are commonly used?

Answer:
```

chunk_size chunk_overlap separator

```
TEXT

---

## Q3. Does CharacterTextSplitter understand document meaning?

Answer:

No.

It only counts characters and does not understand sentences, paragraphs, or semantic
relationships.

---

## Q4. What is the biggest limitation of CharacterTextSplitter?

Answer:

It can split text in the middle of sentences and paragraphs, leading to loss of context.

---

## Q5. Why is overlap important?

Answer:

Overlap preserves context between adjacent chunks and reduces information loss at chunk
boundaries.
```

```
---

## Q6. When should CharacterTextSplitter be used?

Answer:

For simple text documents, logs, and smaller datasets where semantic splitting is not
critical.

---

## Q7. What splitter is generally preferred for production RAG systems?

Answer:
```

RecursiveCharacterTextSplitter

```
TEXT

because it preserves document structure more effectively.

---

# Interview Takeaway

CharacterTextSplitter is the simplest text splitter in LangChain. It divides documents based
 purely on character count using configurable chunk_size and chunk_overlap values. While it
is fast and easy to use, it lacks awareness of sentences, paragraphs, and semantic
structure, which can lead to poor chunk quality. For this reason, it is useful for simple
ingestion scenarios but is generally replaced by RecursiveCharacterTextSplitter in modern
production-grade RAG systems.


# RecursiveCharacterTextSplitter

## What is RecursiveCharacterTextSplitter?

RecursiveCharacterTextSplitter is the most popular and most widely used text splitter in
LangChain.

It is considered the default splitter for most RAG applications because it attempts to
preserve the natural structure of text while creating chunks.

Unlike CharacterTextSplitter, which simply counts characters, RecursiveCharacterTextSplitter
 tries to split text intelligently.

It follows a hierarchy:
```

Paragraph v Sentence v Word v Character

```
TEXT

This hierarchy helps maintain semantic meaning and context.

---

# Why Do We Need RecursiveCharacterTextSplitter?

Consider a document:
```

Password Policy

Passwords must be changed every 90 days.

Multi-Factor Authentication is mandatory.

Privileged access requires approval.

```
TEXT

CharacterTextSplitter might split:
```

Passwords must be changed every 90 day

s. Multi-Factor Authentication is...

```
TEXT

Meaning gets broken.

RecursiveCharacterTextSplitter tries to avoid this.

Preferred output:
```

Chunk 1

Password Policy

Passwords must be changed every 90 days.

```
TEXT
```

Chunk 2

Multi-Factor Authentication is mandatory.

Privileged access requires approval.

```
TEXT

This leads to much better retrieval quality.

---

# Why Is It Called "Recursive"?

The splitter tries multiple separators recursively.

Default separator hierarchy:
```

["\n\n", "\n", " ", ""]

```
TEXT

Meaning:

### First Attempt

Split by paragraphs.
```

Paragraph 1

Paragraph 2

Paragraph 3

```
TEXT

---

### If Chunk Is Still Too Large

Split by line breaks.
```

Line 1 Line 2 Line 3

```
TEXT

---

### If Still Too Large

Split by spaces.
```

Word1 Word2 Word3

```
TEXT

---

### Final Attempt

Split by characters.
```

A B C D E

```
TEXT

This recursive fallback mechanism is why it is called RecursiveCharacterTextSplitter.

---

# Why Is It Preferred for RAG?

Most enterprise documents contain:
```

Paragraphs Sections Policies Descriptions Procedures

```
TEXT

RAG systems need chunks that preserve:

[OK] Context

[OK] Meaning

[OK] Section Boundaries

[OK] Topic Boundaries

RecursiveCharacterTextSplitter provides this.

---

# Where Does It Fit in RAG?
```

Documents v Parsing v Cleaning v Normalization v RecursiveCharacterTextSplitter v Chunks v Embeddings v Vector Database v Retriever v LLM

```
TEXT

---

# Import
```

from langchain_text_splitters import RecursiveCharacterTextSplitter

```
TEXT

---

# Basic Example
```

from langchain_text_splitters import RecursiveCharacterTextSplitter

text = """ Password Policy

Passwords must be changed every 90 days.

Multi-Factor Authentication is mandatory.

VPN access requires approval. """

splitter = RecursiveCharacterTextSplitter(chunk_size=100, chunk_overlap=20)

chunks = splitter.split_text(text)

for chunk in chunks: print(chunk)

```
TEXT

---

# Output Concept

Chunk 1
```

Password Policy

Passwords must be changed every 90 days.

```
TEXT

Chunk 2
```

Multi-Factor Authentication is mandatory.

VPN access requires approval.

```
TEXT

Notice:

[OK] Better chunk boundaries

[OK] Better readability

[OK] Better semantic grouping

---

# Core Parameters

## chunk_size
```

```
Maximum size of a chunk.

Example:
```

chunk_size=1000

```
TEXT

Means:
```

Approximately 1000 characters per chunk

```
TEXT

---

## chunk_overlap

Amount of overlap between chunks.

Example:
```

chunk_overlap=100

```
TEXT

Meaning:
```

Last 100 characters v Copied into next chunk

```
TEXT

Preserves context.

---

## separators

Custom separator hierarchy.

Default:
```

["\n\n", "\n", " ", ""]

```
TEXT

Example:
```

splitter = RecursiveCharacterTextSplitter(separators=["\n\n", "\n", ".", " ", ""])

```
TEXT

Now periods are considered.

---

# Internal Working

Suppose:
```

5000 Character Document

```
TEXT

Chunk Size:
```

1000

```
TEXT

The splitter tries:
```

Paragraph Split

```
TEXT

If chunk still exceeds:
```

1000 Characters

```
TEXT

Try:
```

Line Split

```
TEXT

If still exceeds:
```

Word Split

```
TEXT

If still exceeds:
```

Character Split

```
TEXT

Result:
```

Meaning preserved as much as possible.

```
TEXT

---

# Example of Recursive Splitting

Document:
```

Section 1

Paragraph A

Paragraph B

Paragraph C

```
TEXT

Desired:
```

chunk_size=200

```
TEXT

Process:
```

Try Paragraph Boundaries

```
TEXT

If:
```

Paragraph A + B

```
TEXT

fits:
```

Keep together

```
TEXT

Otherwise:
```

Split further

```
TEXT

---

# Using With Documents

Most production systems use:
```

split_documents()

```
TEXT

Example:
```

from langchain_text_splitters import (RecursiveCharacterTextSplitter)

chunks = splitter.split_documents(documents)

```
TEXT

Input:
```

Document(page_content="...")

```
TEXT

Output:
```

Document(page_content="Chunk...")

```
TEXT

Metadata is preserved.

---

# Example with PDF Loader
```

from langchain_community.document_loaders import PyPDFLoader

```
from langchain_text_splitters import ( RecursiveCharacterTextSplitter )
loader = PyPDFLoader( "security_policy.pdf" )
docs = loader.load()
splitter = RecursiveCharacterTextSplitter( chunk_size=1000, chunk_overlap=100 )
chunks = splitter.split_documents( docs )
```

```
TEXT
Pipeline:
```

PDF v PyPDFLoader v Documents v RecursiveCharacterTextSplitter v Chunks

```
TEXT
---
# Metadata Preservation
Original:
```

{ "source":"policy.pdf", "page":10 }

```
TEXT
After splitting:
```

{ "source":"policy.pdf", "page":10 }

```
TEXT
Metadata remains attached.
This is extremely important for:
[OK] Source Attribution
[OK] Citations
[OK] Traceability
---
# Example Without Overlap
Configuration:
```

chunk_size=100 chunk_overlap=0

```
TEXT
Output:
```

Chunk 1

Password Policy

```
TEXT
```

Chunk 2

MFA Policy

```
TEXT

Potential context loss.

---

# Example With Overlap

Configuration:
```

chunk_size=100 chunk_overlap=20

```
TEXT

Output:
```

Chunk 1

Password Policy MFA Policy

```
TEXT
```

Chunk 2

MFA Policy VPN Policy

```
TEXT

Some context repeated.

---

# Why Overlap Matters

User asks:
```

What

TokenTextSplitter

What is TokenTextSplitter?

TokenTextSplitter is a LangChain text splitter that divides text based on tokens instead of characters.

Unlike:

```
PYTHON
CharacterTextSplitter
```

which counts characters,

and

```
PYTHON
RecursiveCharacterTextSplitter
```

which uses separators,

TokenTextSplitter splits content according to the number of tokens that an LLM tokenizer produces.

This makes it much more aligned with how LLMs process text.

Why Do We Need TokenTextSplitter?

LLMs do not understand:

```
TEXT
Characters
```

LLMs process:

```
TEXT
Tokens
```

Example:

Text:

```
TEXT
Passwords must be changed every 90 days.
```

May become:

```
TEXT
[
 "Passwords",
 "must",
 "be",
 "changed",
 "every",
 "90",
 "days",
 "."
]
```

8 tokens.

Embedding models and LLMs have token limits.

Examples:

```
TEXT
8192 Tokens

16384 Tokens

32000 Tokens

128000 Tokens
```

To stay within these limits, chunking should ideally be token-based.

Character vs Token Splitting

Character-Based

```
TEXT
1000 Characters
```

Problem:

```
TEXT
1000 characters
!=
1000 tokens
```

Different languages produce different token counts.

Token-Based

```
TEXT
1000 Tokens
```

Advantage:

```
TEXT
Matches how the model processes text.
```

This provides more predictable chunk sizes.

Why Is Token Splitting Important for LLMs?

Suppose:

```
TEXT
Chunk Size = 1000 Characters
```

Document A:

```
TEXT
Mostly short words
```

May produce:

```
TEXT
200 Tokens
```

Document B:

```
TEXT
Technical Text
```

May produce:

```
TEXT
500 Tokens
```

Character count remains the same.

Token count differs significantly.

TokenTextSplitter solves this issue.

Where Does TokenTextSplitter Fit In RAG?

```text
Document
v
Parsing
v
Cleaning
v
Normalization
v
TokenTextSplitter
v
Chunks
v
Embeddings
v
Vector Database
v
Retriever
v
LLM
```

Import

```python
from langchain_text_splitters import TokenTextSplitter
```

Installation

```bash
pip install tiktoken
```

TokenTextSplitter commonly uses:

```text
tiktoken
```

for token counting.

Basic Example

```python
from langchain_text_splitters import TokenTextSplitter

text = """
Passwords must be changed every 90 days.

Multi-factor authentication is required.

VPN access requires approval.
"""

splitter = TokenTextSplitter(
    chunk_size=20,
    chunk_overlap=5
```

```
)

chunks = splitter.split_text(text)

print(chunks)
```

Core Parameters

chunk_size

Represents:

```
TEXT
Maximum Tokens Per Chunk
```

Example:

```
PYTHON
chunk_size=1000
```

means:

```
TEXT
1000 Tokens
```

NOT:

```
TEXT
1000 Characters
```

chunk_overlap

Represents:

```
TEXT
Token Overlap Between Chunks
```

Example:

```
PYTHON
chunk_overlap=100
```

means:

```
TEXT
Last 100 tokens
v
Repeated in next chunk
```

Internal Working

Document:

```
TEXT
2000 Tokens
```

Configuration:

```
PYTHON
chunk_size=500

chunk_overlap=50
```

Chunking:

```
TEXT
Token 1-500

Token 451-950

Token 901-1400

Token 1351-1850

Token 1801-2000
```

Notice:

```
TEXT
50 Token Overlap
```

between chunks.

Example Using Documents

```
PYTHON
from langchain_text_splitters import TokenTextSplitter

splitter = TokenTextSplitter(
    chunk_size=500,
    chunk_overlap=50
)

chunks = splitter.split_documents(
    documents
)
```

Input:

```
PYTHON
Document(
    page_content="..."
)
```

Output:

```
PYTHON
Document(
    page_content="Chunk..."
)
```

Metadata remains preserved.

Why Token Splitting Is Better For LLM Limits

Suppose a model supports:

```
TEXT
8192 Tokens
```

Using:

```
PYTHON
chunk_size=1000
```

allows predictable sizing.

Character splitting cannot guarantee this.

Token splitting can.

TokenTextSplitter with OpenAI Models

OpenAI models internally work with tokens.

Example:

```
TEXT
GPT Models

Embedding Models
```

Using TokenTextSplitter provides chunk sizes aligned with model limitations.

TokenTextSplitter with Embedding Models

Embedding Model:

```
TEXT
Input Text
v
Tokens
v
Embedding Vector
```

Because embeddings are token-based, token chunking often produces more consistent vectors.

Example: Why Token Count Matters

Text A:

```
TEXT
Security Policy
```

Characters:

```
TEXT
15
```

Tokens:

```
TEXT
2
```

Text B:

```
TEXT
Multi-factor-authentication-configuration-settings
```

Characters:

```
TEXT
48
```

Tokens:

May become:

```
TEXT
8-12 Tokens
```

depending on tokenizer.

This is why character count can be misleading.

Advantages

1. Token-Aware

Matches LLM processing.

2. Better Context Window Management

Works directly with model limits.

3. Predictable Chunk Sizes

More accurate than character counting.

4. Better Embedding Stability

Consistent token distribution.

5. Production Friendly

Frequently used in enterprise systems.

Limitations

1. No Semantic Awareness

Still doesn't understand

Markdown Splitter (MarkdownHeaderTextSplitter)

What is Markdown Splitter?

Markdown Splitter is a LangChain text splitter specifically designed for Markdown documents.

Instead of splitting purely by:

- Characters
- Tokens
- Fixed Sizes

it splits documents using Markdown structure.

Examples:

```
MARKDOWN
# Title

## Section

### Subsection
```

It understands document hierarchy and preserves document organization during chunking.

This makes it ideal for:

- Technical Documentation
- README Files
- Wikis
- Knowledge Bases
- API Documentation
- Developer Guides

Why Do We Need Markdown Splitter?

Consider a document:

```
MARKDOWN
# Authentication

Authentication is required before using APIs.

## JWT Authentication

JWT tokens are supported.

## OAuth Authentication

OAuth 2.0 is also supported.

# Deployment

Deploy using Docker.
```

If we use CharacterTextSplitter:

```
TEXT
Authentication

Authentication is required before using APIs.

JWT Authenti
```

Meaning can be broken.

Markdown Splitter preserves structure:

Chunk 1

```
MARKDOWN
# Authentication

Authentication is required before using APIs.
```

Chunk 2

```
MARKDOWN
## JWT Authentication

JWT tokens are supported.
```

Chunk 3

```
MARKDOWN
## OAuth Authentication

OAuth 2.0 is also supported.
```

Chunk 4

```
MARKDOWN
# Deployment

Deploy using Docker.
```

Much better for retrieval.

Why Is Markdown Common in RAG Systems?

A lot of enterprise knowledge exists as:

```
TEXT
README.md

Wiki Pages

Confluence Exports

Technical Documents

Product Documentation

API Documentation

Knowledge Bases
```

Most parsing systems must preserve section boundaries.

MarkdownHeaderTextSplitter helps achieve this.

Where Does Markdown Splitter Fit In RAG?

```
TEXT
Markdown Document
v
MarkdownHeaderTextSplitter
v
Structured Chunks
v
Embeddings
v
Vector Database
v
Retriever
v
LLM
```

Import

```
PYTHON
from langchain_text_splitters import (
    MarkdownHeaderTextSplitter
)
```

Basic Markdown Example

Document:

```
MARKDOWN
# Security Policy

Passwords must change every 90 days.

## MFA

Multi-factor authentication is required.

## VPN

VPN access requires approval.
```

Basic Example

```python
from langchain_text_splitters import (
    MarkdownHeaderTextSplitter
)

headers_to_split_on = [
    ("#", "Header 1"),
    ("##", "Header 2"),
]

splitter = MarkdownHeaderTextSplitter(
    headers_to_split_on=headers_to_split_on
)

chunks = splitter.split_text(markdown_text)
```

Understanding headers_to_split_on

Example:

```python
headers_to_split_on = [

    ("#", "Header 1"),

    ("##", "Header 2"),

    ("###", "Header 3")
]
```

Meaning:

```text
Split when:
# appears

Split when:
## appears

Split when:
### appears
```

The splitter captures document hierarchy.

Output Structure

Input:

```
MARKDOWN
# Authentication

JWT Authentication

## OAuth

OAuth Details

# Deployment

Docker Deployment
```

Output:

```
PYTHON
Document(
    page_content="JWT Authentication",
    metadata={
        "Header 1":"Authentication"
    }
)

Document(
    page_content="OAuth Details",
    metadata={
        "Header 1":"Authentication",
        "Header 2":"OAuth"
    }
)

Document(
    page_content="Docker Deployment",
    metadata={
        "Header 1":"Deployment"
    }
)
```

Notice:

[OK] Section metadata automatically created.

[OK] Structure preserved.

[OK] Retrieval becomes more accurate.

Metadata Generation

One of the biggest advantages of Markdown splitting is metadata creation.

Example:

```
MARKDOWN
# Authentication

## JWT

JWT Details
```

Metadata:

```
PYTHON
{
    "Header 1":"Authentication",
    "Header 2":"JWT"
}
```

This metadata can later help:

- Filtering
- Source Attribution
- Navigation
- Explainability

Why Metadata Matters?

User asks:

```
TEXT
Show JWT Authentication details.
```

Retriever can use:

```
PYTHON
{
    "Header 1":"Authentication"
}
```

to find relevant chunks quickly.

Handling Multiple Header Levels

Example:

```
MARKDOWN
# Product

## Authentication

### JWT

JWT Details

### OAuth

OAuth Details
```

Configuration:

```
PYTHON
headers_to_split_on = [
    ("#", "H1"),
    ("##", "H2"),
    ("###", "H3")
]
```

Output Metadata:

```
PYTHON
{
    "H1":"Product",
    "H2":"Authentication",
    "H3":"JWT"
}
```

Very useful for hierarchical retrieval.

Internal Working

Document:

```
MARKDOWN
# Security

Text A

## Passwords

Text B

## MFA

Text C
```

Process:

```
TEXT
Read Header

Store Metadata

Extract Text

Create Document

Repeat
```

Result:

```
TEXT
Security Chunk

Passwords Chunk

MFA Chunk
```

instead of one giant document.

Markdown Splitter vs Character Splitter

CharacterTextSplitter

Splits:

```
TEXT
By Character Count
```

Pros:

[OK] Simple

Cons:

[X] Breaks sections

[X] Breaks headings

Markdown Splitter

Splits:

```
TEXT
By Document Structure
```

Pros:

[OK] Preserves sections

[OK] Better retrieval

[OK] Better metadata

Cons:

[X] Works only with markdown

Markdown Splitter vs RecursiveCharacterTextSplitter

RecursiveCharacterTextSplitter

Uses:

```
TEXT
Paragraphs
Lines
Words
```

Pros:

[OK] General purpose

Cons:

[X] Doesn't understand markdown hierarchy

MarkdownHeaderTextSplitter

Uses:

```
TEXT
Headers
Sections
Subsections
```

Pros:

[OK] Keeps document structure

[OK] Better documentation retrieval

Combining Markdown Splitter with Recursive Splitter

Very common in production.

Step 1:

```
TEXT
Split by Markdown Headers
```

Step 2:

```
TEXT
Split large sections recursively
```

Pipeline:

```
TEXT
Markdown
v
MarkdownHeaderTextSplitter
v
Large Sections
v
RecursiveCharacterTextSplitter
v
Final Chunks
```

Best of both approaches.

Enterprise Example

Documentation:

```
MARKDOWN
# Authentication

## JWT

## OAuth

# Deployment

## Docker

## Kubernetes

# Monitoring

## Logging

## Metrics
```

Markdown Splitter creates:

```
TEXT
Authentication Chunk

JWT Chunk

OAuth Chunk

Deployment Chunk

Docker Chunk

Kubernetes Chunk

Monitoring Chunk
```

Much cleaner retrieval.

Common Use Cases

API Documentation

Examples:

```
TEXT
REST APIs

GraphQL APIs
```

Internal Wikis

Examples:

```
TEXT
Confluence

Developer Wiki
```

README Files

Examples:

```
TEXT
GitHub Projects

Open Source Documentation
```

Product Documentation

Examples:

```
TEXT
User Guides

Installation Guides
```

Technical Knowledge Bases

Examples:

```
TEXT
Architecture Docs

Design Documents
```

Advantages

1. Structure-Aware

Understands headers.

2. Better Retrieval

Sections stay together.

3. Metadata Generation

Headers become metadata.

4. Better Semantic Meaning

Chunks align with document organization.

5. Ideal for Documentation

Perfect for technical docs.

Limitations

1. Markdown Only

Not useful for:

```
TEXT
PDF
DOCX
CSV
```

2. Depends on Good Headers

Poorly formatted markdown reduces effectiveness.

3. Large Sections Still Require Additional Splitting

Often combined with RecursiveCharacterTextSplitter.

Best Practices

[OK] Split Using Header Structure First

[OK] Preserve Header Metadata

[OK] Combine With Recursive Splitting

[OK] Use for Documentation

[OK] Keep Hierarchical Relationships

[OK] Validate Generated Sections

Common Interview Questions

Q1. What is MarkdownHeaderTextSplitter?

Answer:

MarkdownHeaderTextSplitter is a LangChain text splitter that divides Markdown documents based on header hierarchy rather than character counts.

Q2. Why use Markdown Splitter instead of CharacterTextSplitter?

Answer:

Because it preserves document structure, headings, and section boundaries, producing better chunks for retrieval.

Q3. What parameter controls splitting?

Answer:

```
PYTHON
headers_to_split_on
```

Q4. What metadata does Markdown Splitter generate?

Answer:

Header values become metadata.

Example:

```
PYTHON
{
    "Header 1":"Authentication",
    "Header 2":"JWT"
}
```

Q5. Can Markdown Splitter be combined with RecursiveCharacterTextSplitter?

Answer:

Yes. This is a common production pattern where documents are first split by headers and then recursively split into smaller chunks.

Q6. When should Markdown Splitter be used?

Answer:

For documentation, wikis, README files, API guides, and technical knowledge bases written in Markdown.

Q7. What is the biggest advantage of Markdown Splitter?

Answer:

It preserves document hierarchy and semantic structure while creating meaningful metadata.

Interview Takeaway

MarkdownHeaderTextSplitter is a structure-aware LangChain text splitter designed specifically for Markdown documents. Instead of splitting by character count, it uses Markdown headers to preserve document hierarchy, generate useful metadata, and create semantically meaningful chunks. It is widely used for technical documentation, API references, knowledge bases, README files, and enterprise wiki systems. In production RAG applications, it is often combined with RecursiveCharacterTextSplitter to maintain structure while controlling chunk size.

HTML Splitter (HTMLHeaderTextSplitter)

What is HTML Splitter?

HTMLHeaderTextSplitter is a LangChain text splitter specifically designed for HTML documents.

Instead of splitting based on:

- Character Count
- Token Count
- Fixed Size

it uses HTML document structure.

Examples:

```
HTML
<h1>Authentication</h1>

<h2>JWT Authentication</h2>

<h2>OAuth Authentication</h2>

<h1>Deployment</h1>
```

The splitter understands HTML headers and creates chunks based on document hierarchy.

This preserves the semantic structure of HTML content.

Why Do We Need HTML Splitter?

Consider an HTML document:

```HTML
<h1>Authentication</h1>

<p>Authentication is required.</p>

<h2>JWT</h2>

<p>JWT Tokens are supported.</p>

<h2>OAuth</h2>

<p>OAuth 2.0 is supported.</p>

<h1>Deployment</h1>

<p>Deploy using Docker.</p>
```

If we use CharacterTextSplitter:

```TEXT
Authentication is required.

JWT Tokens are supp...
```

Sections may be broken.

HTMLHeaderTextSplitter preserves structure.

Chunk 1

```HTML
Authentication

Authentication is required.
```

Chunk 2

```HTML
JWT

JWT Tokens are supported.
```

Chunk 3

```HTML
OAuth

OAuth 2.0 is supported.
```

Chunk 4

```HTML
Deployment

Deploy using Docker.
```

Why Is HTML Important in RAG?

Most enterprise knowledge exists in:

```
TEXT
Documentation Websites

Help Centers

Knowledge Bases

Internal Portals

Wiki Systems

Developer Documentation

Support Portals
```

These systems often use HTML.

Preserving HTML structure greatly improves retrieval quality.

Where Does HTML Splitter Fit In RAG?

```
TEXT
HTML Document
v
HTMLHeaderTextSplitter
v
Structured Chunks
v
Embeddings
v
Vector Database
v
Retriever
v
LLM
```

Import

```
PYTHON
from langchain_text_splitters import (
    HTMLHeaderTextSplitter
)
```

Basic HTML Example

Document:

```HTML
<h1>Security Policy</h1>

<p>
Passwords must be changed every 90 days.
</p>

<h2>MFA</h2>

<p>
Multi-factor authentication is required.
</p>

<h2>VPN</h2>

<p>
VPN approval is required.
</p>
```

Basic Example

```PYTHON
from langchain_text_splitters import (
    HTMLHeaderTextSplitter
)

headers_to_split_on = [
    ("h1", "Header 1"),
    ("h2", "Header 2"),
]

splitter = HTMLHeaderTextSplitter(
    headers_to_split_on=headers_to_split_on
)

documents = splitter.split_text(
    html_content
)
```

Understanding headers_to_split_on

Example:

```PYTHON
headers_to_split_on = [

    ("h1", "Main Topic"),

    ("h2", "Section"),

    ("h3", "Subsection")
]
```

Meaning:

```
TEXT
Split on:

<h1>

Split on:

<h2>

Split on:

<h3>
```

and preserve hierarchy.

Output Structure

Input:

```
HTML
<h1>Authentication</h1>

<p>Authentication Details</p>

<h2>JWT</h2>

<p>JWT Details</p>

<h2>OAuth</h2>

<p>OAuth Details</p>
```

Output:

```
PYTHON
Document(
    page_content="Authentication Details",
    metadata={
        "Header 1":"Authentication"
    }
)

Document(
    page_content="JWT Details",
    metadata={
        "Header 1":"Authentication",
        "Header 2":"JWT"
    }
)

Document(
    page_content="OAuth Details",
    metadata={
        "Header 1":"Authentication",
        "Header 2":"OAuth"
    }
)
```

Metadata Generation

A major advantage of HTMLHeaderTextSplitter is metadata creation.

Example:

```
HTML
<h1>Authentication</h1>

<h2>JWT</h2>

<p>JWT Details</p>
```

Metadata:

```
PYTHON
{
    "Header 1":"Authentication",
    "Header 2":"JWT"
}
```

Benefits:

[OK] Better Retrieval

[OK] Better Filtering

[OK] Better Navigation

[OK] Better Source Attribution

Multiple Header Levels

Example:

```
HTML
<h1>Product</h1>

<h2>Authentication</h2>

<h3>JWT</h3>

<p>JWT Details</p>
```

Configuration:

```
PYTHON
headers_to_split_on = [
    ("h1", "H1"),
    ("h2", "H2"),
    ("h3", "H3")
]
```

Metadata:

```
PYTHON
{
    "H1":"Product",
    "H2":"Authentication",
    "H3":"JWT"
}
```

Very useful in large documentation websites.

Internal Working

Document:

```
HTML
<h1>Security</h1>

Text A

<h2>Passwords</h2>

Text B

<h2>MFA</h2>

Text C
```

Process:

```
TEXT
Read Header
v
Capture Metadata
v
Extract Content
v
Create Document
v
Repeat
```

Output:

```
TEXT
Security Chunk

Passwords Chunk

MFA Chunk
```

HTML Splitter vs CharacterTextSplitter

CharacterTextSplitter

Splits:

```
TEXT
By Character Count
```

Pros:

[OK] Simple

Cons:

[X] Breaks Sections

[X] Ignores HTML Structure

HTMLHeaderTextSplitter

Splits:

```
TEXT
By HTML Structure
```

Pros:

[OK] Preserves Headings

[OK] Preserves Sections

[OK] Better Retrieval

[OK] Better Metadata

HTML Splitter vs RecursiveCharacterTextSplitter

RecursiveCharacterTextSplitter

Uses:

```
TEXT
Paragraphs

Lines

Words
```

Pros:

[OK] General Purpose

Cons:

[X] Doesn't understand HTML hierarchy

HTMLHeaderTextSplitter

Uses:

```
TEXT
HTML Headings

h1

h2

h3
```

Pros:

[OK] Structure-Aware

[OK] Better for Documentation Sites

Combining HTML Splitter with Recursive Splitting

Extremely common in enterprise systems.

Step 1

```
TEXT
Split By HTML Headers
```

Step 2

```
TEXT
Recursively Split Large Sections
```

Pipeline:

```
TEXT
HTML
v
HTMLHeaderTextSplitter
v
Large Sections
v
RecursiveCharacterTextSplitter
v
Final Chunks
```

Best Practice:

```
TEXT
Preserve Structure
+
Control Chunk Size
```

Example: Documentation Website

HTML:

```
HTML
<h1>Authentication</h1>

<h2>JWT</h2>

<h2>OAuth</h2>

<h1>Deployment</h1>

<h2>Docker</h2>

<h2>Kubernetes</h2>
```

Generated Chunks:

```
TEXT
Authentication

JWT

OAuth

Deployment

Docker

Kubernetes
```

Retriever can find much more precise information.

Enterprise Use Cases

Documentation Websites

Examples:

```
TEXT
Developer Portals

API Documentation

Technical Guides
```

Knowledge Bases

Examples:

```
TEXT
Internal Wikis

Support Centers

Policy Portals
```

E-Learning Platforms

Examples:

```
TEXT
Training Documents

Learning Material
```

Product Support Systems

Examples:

```
TEXT
FAQs

Troubleshooting Guides

User Manuals
```

Advantages

1. Preserves Document Structure

Maintains HTML hierarchy.

2. Better Retrieval Quality

Sections remain logically grouped.

3. Automatic Metadata Creation

Headers become metadata.

4. Ideal for Websites

Especially documentation websites.

5. Improves Semantic Search

Related content stays together.

Limitations

1. HTML Only

Not suitable for:

```
TEXT
PDF

DOCX

CSV
```

2. Depends on Proper HTML Structure

Poor HTML reduces effectiveness.

3. Large Sections Need Further Splitting

Often combined with RecursiveCharacterTextSplitter.

Best Practices

[OK] Split By Header Hierarchy

[OK] Preserve Header Metadata

[OK] Use h1, h2, h3 Structure

[OK] Combine With Recursive Splitting

[OK] Validate Generated Chunks

[OK] Preserve Parent-Child Relationships

Common Interview Questions

Q1. What is HTMLHeaderTextSplitter?

Answer:

HTMLHeaderTextSplitter is a LangChain splitter that creates chunks using HTML header tags such as h1, h2, and h3 rather than character counts.

Q2. Why use HTML Splitter instead of CharacterTextSplitter?

Answer:

Because it preserves website structure, section boundaries, and semantic organization, leading to better retrieval quality.

Q3. Which HTML tags are commonly used?

Answer:

```HTML
<h1>

<h2>

<h3>

<h4>
```

Q4. What metadata does HTML Splitter generate?

Answer:

Header values become metadata.

Example:

```PYTHON
{
    "Header 1":"Authentication",
    "Header 2":"JWT"
}
```

Q5. Can HTML Splitter be combined with RecursiveCharacterTextSplitter?

Answer:

Yes. This is one of the most common production patterns for handling large documentation websites.

Q6. When should HTMLHeaderTextSplitter be used?

Answer:

For HTML documents, documentation websites, knowledge bases, help centers, internal portals, and API documentation.

Q7. What is the biggest advantage of HTML Splitter?

Answer:

It preserves HTML hierarchy and generates valuable metadata that improves retrieval and navigation.

Interview Takeaway

HTMLHeaderTextSplitter is a structure-aware LangChain text splitter designed specifically for HTML documents. It uses HTML header tags such as h1, h2, and h3 to preserve document hierarchy, generate metadata, and create semantically meaningful chunks. It is commonly used for documentation websites, developer portals, support centers, and enterprise knowledge bases. In production RAG systems, it is often combined with RecursiveCharacterTextSplitter to retain document structure while enforcing chunk size limits.

Code Splitter

What is Code Splitter?

Code Splitter is a specialized text splitter in LangChain designed for source code files.

Unlike:

- CharacterTextSplitter
- RecursiveCharacterTextSplitter
- TokenTextSplitter

which treat code as plain text,

Code Splitter understands programming language structure and creates chunks based on code syntax.

Examples:

```
PYTHON
Functions

Classes

Methods

Imports

Modules

Interfaces
```

Instead of splitting in the middle of a function, it tries to preserve logical code boundaries.

This is extremely important in:

- Code RAG Systems
- GitHub Repository Chatbots
- AI Coding Assistants
- Developer Copilots
- Code Search Engines

Why Do We Need Code Splitter?

Suppose we have:

```
PYTHON
class UserService:

    def login(self):
        pass

    def logout(self):
        pass

    def reset_password(self):
        pass
```

CharacterTextSplitter may produce:

Chunk 1:

```
PYTHON
class UserService:

    def login(self):
```

Chunk 2:

```
PYTHON
pass

    def logout(self):
```

This breaks code meaning.

Code Splitter tries to preserve:

```
PYTHON
login()
```

as a complete unit.

Much better for retrieval.

Why Is Code Chunking Different?

Normal text:

```
TEXT
Paragraphs

Sentences

Words
```

Code:

```
TEXT
Functions

Classes

Methods

Modules

Packages
```

A retriever should retrieve:

```
PYTHON
def authenticate_user():
```

rather than a random portion of code.

Therefore code-aware chunking is required.

Where Does Code Splitter Fit In RAG?

```
TEXT
Source Code
v
Code Splitter
v
Code Chunks
v
Embeddings
v
Vector Database
v
Retriever
v
LLM
```

Supported Languages

LangChain supports multiple languages.

Examples:

```
TEXT
Python

Java

JavaScript

TypeScript

C++

C#

Go
```

```
PHP

Rust

Kotlin
```

Import

```
PYTHON
from langchain_text_splitters import (
    RecursiveCharacterTextSplitter,
    Language
)
```

Basic Example

Python File:

```
PYTHON
def login():
    pass

def logout():
    pass

def reset_password():
    pass
```

Code:

```
PYTHON
from langchain_text_splitters import (
    RecursiveCharacterTextSplitter,
    Language
)

splitter = RecursiveCharacterTextSplitter.from_language(
    language=Language.PYTHON,
    chunk_size=200,
    chunk_overlap=20
)

chunks = splitter.split_text(code)
```

What is Language-Aware Splitting?

Normal splitting:

```
TEXT
Count Characters
```

Code splitting:

```
TEXT
Understand Python Structure
```

For Python:

```
PYTHON
class

def

imports
```

become natural splitting points.

Supported Language Enum

Example:

```
PYTHON
Language.PYTHON

Language.JAVA

Language.JS

Language.TS

Language.CPP

Language.GO
```

Python Code Example

Original:

```
PYTHON
class UserService:

    def login(self):
        pass

    def logout(self):
        pass


class PaymentService:

    def pay(self):
        pass
```

Code Splitter may generate:

Chunk 1

```
PYTHON
class UserService

login()

logout()
```

Chunk 2

```
PYTHON
class PaymentService

pay()
```

Instead of breaking functions arbitrarily.

Java Example

Code:

```
JAVA
public class UserService {

    public void login() {

    }

    public void logout() {

    }
}
```

Language-Aware Splitter:

```
PYTHON
splitter = RecursiveCharacterTextSplitter.from_language(
    language=Language.JAVA
)
```

Understands:

```
JAVA
class

method

interface
```

boundaries.

JavaScript Example

Code:

```
JAVASCRIPT
function login() {

}

function logout() {

}
```

Splitter:

```
PYTHON
splitter = RecursiveCharacterTextSplitter.from_language(
    language=Language.JS
)
```

Preserves function boundaries.

Internal Working

Suppose:

```
PYTHON
class A

function x()

function y()

class B

function z()
```

Process:

```
TEXT
Identify Language
v
Identify Structural Boundaries
v
Create Logical Chunks
v
Apply Chunk Size Rules
v
Generate Chunks
```

Why Code Structure Matters?

User asks:

```
TEXT
How is password reset implemented?
```

Retriever should return:

```
PYTHON
def reset_password():
```

not:

```
PYTHON
def logi
```

and:

```
PYTHON
n():
```

split across multiple chunks.

Chunking a Large Repository

Repository:

```
TEXT
src/

auth/

database/

services/

controllers/

models/
```

Pipeline:

```
TEXT
GitHub Repository
v
Code Loader
v
Code Splitter
v
Embeddings
v
Vector Database
v
Coding Assistant
```

Code Splitter vs CharacterTextSplitter

CharacterTextSplitter

Splits:

```
TEXT
Characters
```

Pros:

[OK] Simple

Cons:

[X] Breaks code

[X] Breaks functions

[X] Poor retrieval

Code Splitter

Splits:

```
TEXT
Functions

Classes

Methods
```

Pros:

[OK] Preserves code structure

[OK] Better code retrieval

[OK] Better AI responses

Code Splitter vs RecursiveCharacterTextSplitter

RecursiveCharacterTextSplitter

General-purpose.

Uses:

```
TEXT
Paragraphs

Words

Characters
```

Not code-aware.

Code Splitter

Language-aware.

Uses:

```
TEXT
Functions

Classes

Methods

Interfaces
```

Much better for codebases.

Code Embeddings and Chunking

Suppose:

```
PYTHON
def process_payment():
```

Embedding should represent:

```
TEXT
Payment Processing Logic
```

If a function is split improperly:

```
PYTHON
process_pay
```

and

```
PYTHON
ment()
```

Embedding quality decreases.

Good chunking improves code embeddings significantly.

Parent-Child Code Chunking

Large Function:

```
PYTHON
1000 Lines
```

Strategy:

```
TEXT
Parent Chunk
v
Function
```

```
TEXT
Child Chunk
v
Specific Sections
```

This is commonly used in advanced code RAG systems.

Enterprise Example

Organization:

```
TEXT
500 Repositories

2 Million Lines of Code
```

Pipeline:

```
TEXT
GitHub
v
Repository Loader
v
Code Splitter
v
Code Embeddings
v
Vector Database
v
Developer Assistant
```

Developers ask:

```
TEXT
Where is JWT validation implemented?
```

Retriever returns:

```
PYTHON
validate_jwt()
```

directly.

Common Use Cases

AI Coding Assistants

Examples:

```
TEXT
GitHub Copilot-like Systems
```

Code Search

Examples:

```
TEXT
Enterprise Code Search
```

Developer Chatbots

Examples:

```
TEXT
Repository Q&A Bots
```

Software Documentation

Examples:

```
TEXT
Code-Aware Knowledge Bases
```

Legacy Modernization

Examples:

```
TEXT
Analyze Old Codebases
```

Advantages

1. Preserves Code Structure

Functions remain intact.

2. Better Retrieval

More accurate code search.

3. Better Embeddings

Each chunk represents a logical component.

4. Language-Aware

Understands programming syntax.

5. Essential for Code RAG

Industry standard approach.

Limitations

1. Language Dependency

Requires supported language definitions.

2. Complex Repositories

Very large functions may still need additional chunking.

3. Not Semantic

Understands syntax better than meaning.

4. Multi-Language Projects

May require different splitters per language.

Best Practices

[OK] Use Language-Aware Splitting

```
PYTHON
Language.PYTHON
Language.JAVA
Language.JS
```

[OK] Preserve Functions

Functions should remain complete whenever possible.

[OK] Preserve Classes

Avoid splitting class definitions unnecessarily.

[OK] Use Overlap

Maintain context between chunks.

[OK] Test Retrieval Quality

Verify code search effectiveness.

[OK] Store File Metadata

```python
{
    "file":"auth.py",
    "repository":"project-a"
}
```

Common Interview Questions

Q1. What is a Code Splitter?

Answer:

A Code Splitter is a language-aware text splitter that chunks source code based on programming constructs such as functions, classes, methods, and modules.

Q2. Why can't we use CharacterTextSplitter for code?

Answer:

Because it may split code in the middle of functions, classes, or statements, reducing retrieval and embedding quality.

Q3. How does LangChain support code chunking?

Answer:

Using language-aware splitters created through:

```python
RecursiveCharacterTextSplitter.from_language()
```

Q4. Which languages are commonly supported?

Answer:

- Python
- Java
- JavaScript
- TypeScript
- C++
- Go
- PHP
- Rust

Q5. Why is code chunking important for Code RAG systems?

Answer:

It preserves logical code units, improves embeddings, and enables accurate retrieval of functions, classes, and implementation details.

Q6. What is the biggest advantage of Code Splitter?

Answer:

It preserves programming language structure rather than blindly splitting text.

Q7. Where is Code Splitter commonly used?

Answer:

- GitHub Repository Chatbots
- Developer Assistants
- Enterprise Code Search
- Software Documentation Systems
- Code Intelligence Platforms

Interview Takeaway

Code Splitter is a language-aware text splitting strategy used for source code ingestion in LangChain. Unlike generic text splitters, it preserves logical code structures such as functions, classes, methods, and modules, making it ideal for code search, repository Q&A systems, developer assistants, and Code RAG applications. By maintaining code boundaries, it significantly improves embedding quality, retrieval accuracy, and AI-generated code understanding in enterprise software development environments.

Semantic Chunking

What is Semantic Chunking?

Semantic Chunking is an advanced chunking technique that splits documents based on meaning and context rather than:

- Characters
- Tokens
- Paragraph Boundaries
- Fixed Sizes

The goal is:

```
TEXT
Keep related ideas together
Split when topic changes
```

Unlike traditional chunking methods that use arbitrary sizes, Semantic Chunking attempts to identify natural semantic boundaries in the content.

Why Do We Need Semantic Chunking?

Consider a document:

```
TEXT
Password Policy

Passwords must be changed every 90 days.

MFA Policy

Multi-factor authentication is required.

VPN Policy

VPN access requires manager approval.
```

Traditional chunking might produce:

Chunk 1

```
TEXT
Password Policy

Passwords must be changed every 90 days.

MFA Poli
```

Chunk 2

```
TEXT
cy

Multi-factor authentication is required.
```

Notice:

[X] Topic split in the middle

[X] Meaning broken

[X] Lower retrieval accuracy

Semantic Chunking creates:

Chunk 1

```
TEXT
Password Policy

Passwords must be changed every 90 days.
```

Chunk 2

```
TEXT
MFA Policy

Multi-factor authentication is required.
```

Chunk 3

```
TEXT
VPN Policy

VPN access requires manager approval.
```

[OK] Topic preserved

[OK] Better embeddings

[OK] Better retrieval

Why Is Semantic Chunking Important?

RAG systems retrieve chunks.

Embeddings represent chunks.

If chunks contain multiple unrelated topics:

```
TEXT
Password Policy

Vacation Policy

Salary Policy
```

The embedding becomes "confused."

The vector represents multiple concepts simultaneously.

Semantic chunking avoids this problem.

Traditional Chunking vs Semantic Chunking

Traditional Chunking

Based on:

```
TEXT
Characters

Tokens

Paragraphs
```

Example:

```
PYTHON
chunk_size=1000
```

Pros:

[OK] Fast

[OK] Simple

Cons:

[X] Ignores meaning

[X] Can split topics

Semantic Chunking

Based on:

```
TEXT
Meaning

Context

Topic Boundaries
```

Pros:

[OK] Better chunk quality

[OK] Better retrieval

[OK] Better embeddings

Cons:

[X] More expensive

[X] More processing

Where Does Semantic Chunking Fit In RAG?

```
TEXT
Document
v
Parsing
v
Cleaning
v
Normalization
v
Semantic Chunking
v
Embeddings
v
Vector Database
v
Retriever
v
LLM
```

How Semantic Chunking Works

Suppose:

```
TEXT
Topic A

Topic A

Topic A

Topic B

Topic B

Topic C
```

Traditional chunking:

```
TEXT
Fixed Size
```

Semantic chunking:

```
TEXT
Topic A
v
Chunk

Topic B
v
Chunk

Topic C
v
Chunk
```

The splitter tries to identify where the meaning changes.

Internal Working

A simplified workflow:

```
TEXT
Document
v
Sentence Extraction
v
Generate Embeddings
v
Measure Similarity
v
Detect Topic Changes
v
Create Chunks
```

Instead of counting characters:

```
TEXT
Meaning Similarity
```

is measured.

Example

Document:

```
TEXT
Passwords must be changed every 90 days.

Multi-factor authentication is mandatory.

VPN access requires approval.

The company provides health insurance.

Employees receive annual leave benefits.
```

Similarity Analysis:

```
TEXT
Password Policy
v
Related

MFA Policy
v
Related

VPN Policy
v
Related

Health Insurance
v
NEW TOPIC

Annual Leave
v
Related
```

Output:

Chunk 1

```
TEXT
Passwords must be changed every 90 days.

Multi-factor authentication is mandatory.

VPN access requires approval.
```

Chunk 2

```
TEXT
The company provides health insurance.

Employees receive annual leave benefits.
```

Why Embeddings Are Used?

Each sentence can be converted into a vector.

Example:

```
TEXT
Sentence
v
Embedding
v
Vector
```

Similarity between vectors helps determine:

```
TEXT
Same Topic?
OR
New Topic?
```

Semantic Similarity Concept

Example:

Sentence 1

```
TEXT
Passwords must be changed every 90 days.
```

Sentence 2

```
TEXT
Multi-factor authentication is required.
```

High similarity:

```
TEXT
Security Topic
```

Sentence 3

```
TEXT
Employees receive transportation allowance.
```

Low similarity:

```
TEXT
HR Topic
```

Chunk boundary detected.

LangChain Semantic Chunking

Commonly implemented using:

```
PYTHON
SemanticChunker
```

Example:

```
PYTHON
from langchain_experimental.text_splitter import (
    SemanticChunker
)
```

Basic Example

```python
from langchain_experimental.text_splitter import (
    SemanticChunker
)

from langchain_openai import OpenAIEmbeddings

chunker = SemanticChunker(
    OpenAIEmbeddings()
)

documents = chunker.create_documents(
    [text]
)
```

Process:

```text
Text
v
Embeddings
v
Semantic Similarity
v
Chunks
```

Why Is It More Expensive?

Character Splitter:

```text
Count Characters
```

Very cheap.

Semantic Chunker:

```text
Generate Embeddings

Calculate Similarities

Identify Boundaries
```

Requires additional computation.

Semantic Chunking vs RecursiveCharacterTextSplitter

RecursiveCharacterTextSplitter

Uses:

```
TEXT
Paragraphs

Lines

Words

Characters
```

Pros:

[OK] Fast

[OK] Production Ready

[OK] Most Common

Cons:

[X] Doesn't truly understand meaning

Semantic Chunking

Uses:

```
TEXT
Embeddings

Similarity Scores

Topic Changes
```

Pros:

[OK] Better semantic coherence

[OK] Better retrieval

[OK] Better chunk quality

Cons:

[X] Slower

[X] More expensive

Semantic Chunking vs Token Chunking

Token Chunking

Goal:

```
TEXT
Control Token Length
```

Semantic Chunking

Goal:

```
TEXT
Preserve Meaning
```

Token Chunking:

```
TEXT
Model Optimization
```

Semantic Chunking:

```
TEXT
Retrieval Optimization
```

Real Enterprise Example

Suppose a company has:

```
TEXT
5000 Policies

10000 SOPs

15000 Knowledge Articles
```

A single document contains:

```
TEXT
Security

Networking

Compliance

HR

Finance
```

Using fixed chunking:

```
TEXT
Mixed Topics
```

Using semantic chunking:

```
TEXT
Security Chunk

Networking Chunk

Compliance Chunk

HR Chunk

Finance Chunk
```

Retrieval quality improves significantly.

Benefits for Retrieval

User asks:

```
TEXT
What are MFA requirements?
```

Traditional chunk:

```
TEXT
Password Policy
MFA Policy
Insurance Policy
Vacation Policy
```

Semantic chunk:

```
TEXT
Password Policy

MFA Policy
```

Much more relevant.

Benefits for Embeddings

Bad Chunk:

```
TEXT
Security

Payroll

Marketing

Network
```

Single embedding tries to represent everything.

Semantic Chunk:

```
TEXT
Only Security
```

Embedding becomes much more meaningful.

Common Use Cases

Enterprise Knowledge Bases

Examples:

```
TEXT
Policies

Procedures

SOPs
```

Legal Systems

Examples:

```
TEXT
Contracts

Compliance Documents
```

Research Systems

Examples:

```
TEXT
Research Papers

Patents
```

Healthcare

Examples:

```
TEXT
Medical Guidelines

Clinical Documentation
```

Financial Systems

Examples:

```
TEXT
Regulatory Documents

Auditing Standards
```

Advantages

1. Better Semantic Coherence

Related information remains together.

2. Better Retrieval Quality

More accurate search results.

3. Better Embeddings

Vectors represent single topics.

4. Better RAG Responses

Cleaner context improves LLM answers.

5. Reduced Topic Mixing

Prevents unrelated content from being grouped.

Limitations

1. Higher Cost

Requires embeddings during chunking.

2. Slower Processing

More computation than fixed-size chunking.

3. More Complex

Additional tuning often required.

4. Large-Scale Systems Need Optimization

Millions of documents can be expensive to process.

Best Practices

[OK] Use Semantic Chunking for High-Value Content

Examples:

```
TEXT
Policies

Contracts

Research Papers
```

[OK] Evaluate Retrieval Quality

Always compare against RecursiveCharacterTextSplitter.

[OK] Use with Enterprise Knowledge Bases

Particularly where topic boundaries are important.

[OK] Monitor Chunk Sizes

Semantic chunks can sometimes become too large.

[OK] Combine with Token Limits

Ensure chunks remain within model constraints.

Common Interview Questions

Q1. What is Semantic Chunking?

Answer:

Semantic Chunking is a chunking strategy that creates chunks based on meaning and topic boundaries rather than fixed character or token counts.

Q2. How does Semantic Chunking work?

Answer:

It generates embeddings for text segments, measures semantic similarity, detects topic changes, and creates boundaries where meaning shifts.

Q3. Why is Semantic Chunking better than Character Chunking?

Answer:

Because it keeps related concepts together and avoids splitting content in the middle of a topic.

Q4. Why is Semantic Chunking expensive?

Answer:

It requires embedding generation and similarity calculations during the chunking process.

Q5. When should Semantic Chunking be used?

Answer:

For high-value documents such as policies, contracts, technical documentation, research papers, and enterprise knowledge bases where retrieval quality is critical.

Q6. What is the biggest advantage of Semantic Chunking?

Answer:

It produces semantically coherent chunks that significantly improve retrieval accuracy and embedding quality.

Q7. What is the biggest limitation of Semantic Chunking?

Answer:

Higher computational cost and slower ingestion compared to traditional chunking methods.

Interview Takeaway

Semantic Chunking is an advanced chunking technique that groups content based on meaning rather than character counts, token counts, or document formatting. It uses embeddings and semantic similarity to identify topic boundaries and create highly coherent chunks. Although it is more computationally expensive than RecursiveCharacterTextSplitter or TokenTextSplitter, it often produces superior retrieval quality, better embeddings, and more accurate RAG responses, making it an increasingly popular choice for enterprise-grade AI systems.

Chunk Size

What is Chunk Size?

Chunk Size is the maximum amount of content that can exist inside a single chunk during the chunking process.

It determines:

```
TEXT
How large or how small each chunk will be.
```

Chunk Size is one of the most important parameters in any RAG system because it directly impacts:

- Retrieval Quality
- Embedding Quality

- Context Preservation
- Token Usage
- Storage Requirements
- LLM Response Accuracy

Most chunking strategies use a chunk size parameter.

Examples:

```
PYTHON
chunk_size=500
```

```
PYTHON
chunk_size=1000
```

```
PYTHON
chunk_size=1500
```

Why is Chunk Size Important?

Consider a document:

```
TEXT
Security Policy

Passwords must be changed every 90 days.

Multi-factor authentication is required.

VPN access requires manager approval.

Remote desktop access requires approval.

Audit logs are retained for 90 days.
```

The way we choose chunk size determines how content is stored and retrieved.

A poor chunk size can significantly reduce RAG performance.

Where Does Chunk Size Fit in RAG?

```
TEXT
Document
v
Chunking Strategy
v
Chunk Size Configuration
v
Chunks
v
Embeddings
v
Vector Database
v
Retriever
v
LLM
```

Chunk Size is a configuration parameter within the chunking stage.

Small Chunk Size Example

Suppose:

```python
chunk_size = 50
```

Output:

Chunk 1

```text
Passwords must be changed every 90 days.
```

Chunk 2

```text
Multi-factor authentication is required.
```

Chunk 3

```text
VPN access requires approval.
```

Advantages:

[OK] Highly focused chunks

[OK] Precise retrieval

[OK] Smaller embeddings

Disadvantages:

[X] Context loss

[X] Too many chunks

[X] More vector storage

Large Chunk Size Example

Suppose:

```python
chunk_size = 2000
```

Output:

Chunk 1

```text
Security Policy

Passwords must be changed every 90 days.

Multi-factor authentication is required.

VPN access requires approval.

Remote desktop access requires approval.

Audit logs are retained for 90 days.
```

Advantages:

[OK] More context

[OK] Fewer chunks

[OK] Better document continuity

Disadvantages:

[X] More irrelevant information

[X] Retrieval noise

[X] Larger token consumption

Small vs Large Chunk Size

Small Chunks

Example:

```
TEXT
100-300 Tokens
```

Pros:

[OK] Highly specific retrieval

[OK] Lower context noise

[OK] Better precision

Cons:

[X] Less context

[X] May lose relationships

[X] More vectors

Large Chunks

Example:

```
TEXT
1500-3000 Tokens
```

Pros:

[OK] More context

[OK] Better topic continuity

[OK] Fewer chunks

Cons:

[X] Lower retrieval precision

[X] Higher token costs

[X] More irrelevant content returned

Why Chunk Size Impacts Retrieval

Suppose a user asks:

```
TEXT
What are MFA requirements?
```

Document contains:

```
TEXT
Passwords

MFA

VPN

Networking

Auditing

Logging
```

Large Chunk

Retriever returns:

```
TEXT
Passwords

MFA

VPN

Networking

Auditing

Logging
```

Problem:

```
TEXT
Too much unrelated context
```

Smaller Chunk

Retriever returns:

```
TEXT
MFA Requirements
```

Much better precision.

Why Chunk Size Impacts Embeddings

Embeddings represent chunk meaning.

Chunk:

```
TEXT
Password Policy
```

Embedding:

```
TEXT
Focused Security Meaning
```

Chunk:

```
TEXT
Password Policy
Payroll Information
Networking
Insurance
Finance
```

Embedding:

```
TEXT
Mixed Meaning
```

Less effective retrieval.

Chunk Size and Context Preservation

Imagine:

```
TEXT
100 Tokens
```

Chunk Size:

```
PYTHON
50
```

Output:

Chunk 1

```
TEXT
Topic A
```

Chunk 2

```
TEXT
Topic A Continued
```

Meaning may split.

Chunk Size:

```
PYTHON
200
```

Output:

```
TEXT
Entire Topic A
```

Context preserved.

Impact on Vector Database

Suppose:

Document:

```
TEXT
100,000 Tokens
```

Chunk Size:

```
PYTHON
100
```

Results:

```
TEXT
1000 Chunks
```

Chunk Size:

```
PYTHON
1000
```

Results:

```
TEXT
100 Chunks
```

Smaller chunks create:

[OK] Better precision

[X] More storage

Chunk Size and Cost

More Chunks:

```
TEXT
More Embeddings
v
More Storage
v
Higher Cost
```

Fewer Chunks:

```
TEXT
Lower Storage
v
Lower Cost
```

Trade-off always exists.

Typical Chunk Sizes

FAQs

```python
100 - 300
```

Reason:

```text
Questions are already small.
```

Policies

```python
500 - 1000
```

Reason:

```text
Need moderate context.
```

Technical Documentation

```python
800 - 1500
```

Reason:

```text
Concepts often span multiple paragraphs.
```

Research Papers

```python
1000 - 2000
```

Reason:

```text
Complex explanations require larger context.
```

Source Code

```python
300 - 800
```

Reason:

```text
Functions and classes should remain together.
```

Chunk Size with RecursiveCharacterTextSplitter

Example:

```python
splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=100
)
```

Meaning:

```text
Maximum 1000 characters

100 characters overlap
```

Chunk Size with TokenTextSplitter

Example:

```python
splitter = TokenTextSplitter(
    chunk_size=500,
    chunk_overlap=50
)
```

Meaning:

```text
Maximum 500 tokens

50 token overlap
```

Enterprise Example

Suppose a bank has:

```text
Policies
Compliance Documents
Regulations
Audit Standards
```

Total:

```text
50,000 Documents
```

Testing:

```python
Chunk Size = 200
```

Result:

```
TEXT
Excellent Precision

Poor Context
```

Testing:

```
PYTHON
Chunk Size = 3000
```

Result:

```
TEXT
Good Context

Poor Precision
```

Final:

```
PYTHON
Chunk Size = 1000
```

Balanced outcome.

This process is called:

```
TEXT
Chunk Size Tuning
```

Chunk Size Tuning

There is no universal chunk size.

Best chunk size depends on:

```
TEXT
Document Type

Retriever

Embedding Model

Use Case

LLM
```

Typical process:

```
TEXT
Choose Size
v
Test Retrieval
v
Measure Accuracy
v
Adjust Size
v
Repeat
```

Common Mistakes

Mistake 1

Using extremely small chunks.

Example:

```python
chunk_size=20
```

Result:

```text
Context lost.
```

Mistake 2

Using extremely large chunks.

Example:

```python
chunk_size=10000
```

Result:

```text
Poor retrieval precision.
```

Mistake 3

Using same chunk size for all document types.

Policies and source code often need different sizing strategies.

Mistake 4

Ignoring retrieval evaluation.

Chunk size should be measured using actual retrieval performance.

Chunk Size Best Practices

[OK] Start with 500-1000

Good baseline for most RAG applications.

[OK] Tune Based on Documents

Different document types require different sizes.

[OK] Evaluate Retrieval Results

Do not choose sizes blindly.

[OK] Balance Precision and Context

This is the primary goal.

[OK] Consider Embedding Costs

Smaller chunks increase vector count.

[OK] Combine with Overlap

Chunk size and overlap work together.

Chunk Size vs Chunk Overlap

Chunk Size controls:

```
TEXT
How much content enters a chunk.
```

Chunk Overlap controls:

```
TEXT
How much content is repeated between chunks.
```

Example:

```
PYTHON
chunk_size=1000

chunk_overlap=100
```

Means:

```
TEXT
1000 character chunk

100 characters repeated
```

These two parameters are usually tuned together.

Common Interview Questions

Q1. What is Chunk Size?

Answer:

Chunk Size is the maximum amount of content allowed in a chunk during text splitting.

Q2. Why is Chunk Size important?

Answer:

It directly impacts retrieval quality, embedding quality, context preservation, storage requirements, and token consumption.

Q3. What happens when Chunk Size is too small?

Answer:

Context may be lost, and too many chunks are created, increasing storage and embedding costs.

Q4. What happens when Chunk Size is too large?

Answer:

Chunks may contain multiple topics, reducing retrieval precision and increasing irrelevant context.

Q5. Is there a perfect chunk size?

Answer:

No.

The optimal chunk size depends on document type, embedding model, retriever, and business use case.

Q6. What is the most common chunk size for enterprise RAG systems?

Answer:

A common starting point is:

```python
500 - 1000
```

followed by tuning based on retrieval performance.

Q7. Why should chunk size be tested?

Answer:

Because chunk size significantly influences RAG quality and there is no universal value that works for every dataset.

Interview Takeaway

Chunk Size is one of the most critical parameters in a RAG pipeline. It determines how much content is placed inside each chunk before embedding generation and indexing. Small chunk sizes improve retrieval precision but may lose context, while large chunk sizes preserve context but can reduce retrieval accuracy. Successful enterprise RAG systems carefully tune chunk size based on document type, embedding model, retriever behavior, and business requirements to achieve the right balance between context preservation and retrieval precision.

Chunk Overlap

What is Chunk Overlap?

Chunk Overlap is the amount of content that is intentionally repeated between consecutive chunks during the chunking process.

Its purpose is to preserve context that might otherwise be lost when a document is split into multiple chunks.

Example:

Document:

```text
A B C D E F G H I J
```

Without Overlap:

```
TEXT
Chunk 1:
A B C D E

Chunk 2:
F G H I J
```

Notice:

```
TEXT
No shared context
```

With Overlap:

```
TEXT
Chunk 1:
A B C D E

Chunk 2:
D E F G H

Chunk 3:
G H I J
```

Context flows across chunks.

Why Do We Need Chunk Overlap?

When documents are split, important information may lie near chunk boundaries.

Example:

```
TEXT
Multi-Factor Authentication is required for all users.
Authentication tokens expire after 12 hours.
```

Suppose the split happens here:

Chunk 1

```
TEXT
Multi-Factor Authentication is required
```

Chunk 2

```
TEXT
for all users.

Authentication tokens expire after 12 hours.
```

Problem:

```
TEXT
Sentence broken
Context lost
```

With overlap:

Chunk 1

```
TEXT
Multi-Factor Authentication is required for all users.
```

Chunk 2

```
TEXT
required for all users.

Authentication tokens expire after 12 hours.
```

Important information appears in both chunks.

Why Is Overlap Important in RAG?

RAG pipeline:

```
TEXT
Documents
v
Chunking
v
Embeddings
v
Vector Database
v
Retriever
v
LLM
```

If overlap is missing:

```
TEXT
Context near boundaries
v
May disappear
```

If overlap is present:

```
TEXT
Context preserved
v
Better retrieval
v
Better answers
```

Example Without Chunk Overlap

Document:

```
TEXT
Passwords must be changed every 90 days.

Multi-factor authentication is required.

VPN access requires approval.
```

Configuration:

```
PYTHON
chunk_size=50
chunk_overlap=0
```

Output:

Chunk 1

```
TEXT
Passwords must be changed every 90 days.
```

Chunk 2

```
TEXT
Multi-factor authentication is required.
```

Chunk 3

```
TEXT
VPN access requires approval.
```

If information spans chunks:

```
TEXT
Context may be lost.
```

Example With Chunk Overlap

Configuration:

```
PYTHON
chunk_size=50
chunk_overlap=15
```

Output:

Chunk 1

```
TEXT
Passwords must be changed every 90 days.
```

Chunk 2

```
TEXT
every 90 days.

Multi-factor authentication is required.
```

Chunk 3

```
TEXT
authentication is required.

VPN access requires approval.
```

Notice:

[OK] Context preserved

[OK] Retrieval quality improved

Where Does Overlap Fit?

```
TEXT
Document
v
Chunk Size
v
Chunk Overlap
v
Chunks
v
Embeddings
```

Chunk Size and Chunk Overlap are usually configured together.

Example:

```
PYTHON
chunk_size=1000
chunk_overlap=100
```

How Chunk Overlap Works Internally

Document:

```
TEXT
1 2 3 4 5 6 7 8 9 10
```

Configuration:

```
PYTHON
chunk_size=5
chunk_overlap=2
```

Result:

Chunk 1

```
TEXT
1 2 3 4 5
```

Chunk 2

```
TEXT
4 5 6 7 8
```

Chunk 3

```
TEXT
7 8 9 10
```

Repeated content:

```
TEXT
4 5
7 8
```

Overlap in RecursiveCharacterTextSplitter

Example:

```python
from langchain_text_splitters import (
    RecursiveCharacterTextSplitter
)

splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=100
)
```

Meaning:

```text
1000 Characters Per Chunk

100 Characters Repeated
```

between neighboring chunks.

Overlap in TokenTextSplitter

Example:

```python
splitter = TokenTextSplitter(
    chunk_size=500,
    chunk_overlap=50
)
```

Meaning:

```text
500 Tokens Per Chunk

50 Tokens Shared
```

between chunks.

Benefits of Chunk Overlap

1. Context Preservation

Without overlap:

```text
Information gets cut.
```

With overlap:

```text
Context survives chunk boundaries.
```

2. Better Retrieval Accuracy

Retriever can find important content even if it occurs at the edge of a chunk.

3. Better Embeddings

Embeddings contain richer context.

4. Better LLM Responses

The retrieved chunk contains sufficient surrounding information.

5. Reduced Boundary Problems

Prevents losing relationships between nearby sentences.

Example: Retrieval Without Overlap

Document:

```
TEXT
MFA is required for all employees.

Authentication tokens expire after 12 hours.
```

Split:

Chunk 1

```
TEXT
MFA is required
```

Chunk 2

```
TEXT
for all employees.
```

User asks:

```
TEXT
Who needs MFA?
```

Retriever may miss context.

Example: Retrieval With Overlap

Chunk 1

```
TEXT
MFA is required for all employees.
```

Chunk 2

```
TEXT
required for all employees.

Authentication tokens expire after 12 hours.
```

Retriever now has enough information.

Too Little Overlap

Example:

```
PYTHON
chunk_overlap=0
```

Problems:

[X] Context Loss

[X] Boundary Issues

[X] Lower Retrieval Quality

Too Much Overlap

Example:

```
PYTHON
chunk_overlap=800

chunk_size=1000
```

Problems:

[X] Large duplication

[X] More embeddings

[X] More storage

[X] Higher vector database cost

[X] Slower retrieval

Ideal Overlap

Common production values:

Small Chunks

```
PYTHON
chunk_size=300

chunk_overlap=30-50
```

Medium Chunks

```
PYTHON
chunk_size=1000

chunk_overlap=100-200
```

Large Chunks

```
PYTHON
chunk_size=2000

chunk_overlap=200-400
```

Chunk Size vs Chunk Overlap

Chunk Size

Controls:

```
TEXT
How much content exists in a chunk.
```

Example:

```
PYTHON
chunk_size=1000
```

Chunk Overlap

Controls:

```
TEXT
How much content is repeated.
```

Example:

```
PYTHON
chunk_overlap=100
```

Example:

```
PYTHON
chunk_size=1000
chunk_overlap=100
```

Means:

```
TEXT
Chunk Length = 1000

Repeated Content = 100
```

Enterprise Example

Suppose a bank has:

```
TEXT
Compliance Policies

Security Standards

Audit Procedures

Risk Management Documents
```

Without overlap:

```
TEXT
Policy sections get separated.
```

Result:

```
TEXT
Lower retrieval quality
```

With overlap:

```
TEXT
Related sections remain connected.
```

Result:

```
TEXT
Improved retrieval accuracy
```

Impact on Storage

Document:

```
TEXT
100 Pages
```

Overlap:

```
PYTHON
0
```

May create:

```
TEXT
100 Chunks
```

Overlap:

```
PYTHON
200
```

May create:

```
TEXT
120 Chunks
```

Because duplicated content increases chunk count.

Trade-off:

```
TEXT
More Context
vs
More Storage
```

Common Mistakes

Mistake 1

No overlap.

```
PYTHON
chunk_overlap=0
```

Can hurt retrieval.

Mistake 2

Huge overlap.

```
PYTHON
chunk_overlap=500
```

for

```
PYTHON
chunk_size=600
```

Creates excessive duplication.

Mistake 3

Using same overlap for every document type.

Different documents require different settings.

Mistake 4

Never testing retrieval quality.

Overlap should be evaluated using real queries.

Best Practices

[OK] Use Overlap With RecursiveCharacterTextSplitter

Most production systems do this.

[OK] Start With 10-20% Overlap

Example:

```
PYTHON
chunk_size=1000

chunk_overlap=100-200
```

[OK] Tune Using Retrieval Metrics

Measure actual performance.

[OK] Avoid Excessive Duplication

Large overlap increases costs.

[OK] Balance Context and Storage

Choose overlap carefully.

Typical Production Settings

General Documents

```
PYTHON
chunk_size=1000
chunk_overlap=100
```

Technical Documentation

```
PYTHON
chunk_size=1200
chunk_overlap=150
```

Policies

```
PYTHON
chunk_size=800
chunk_overlap=100
```

Source Code

```
PYTHON
chunk_size=500
chunk_overlap=50
```

Common Interview Questions

Q1. What is Chunk Overlap?

Answer:

Chunk Overlap is the amount of content intentionally repeated between consecutive chunks to preserve context across chunk boundaries.